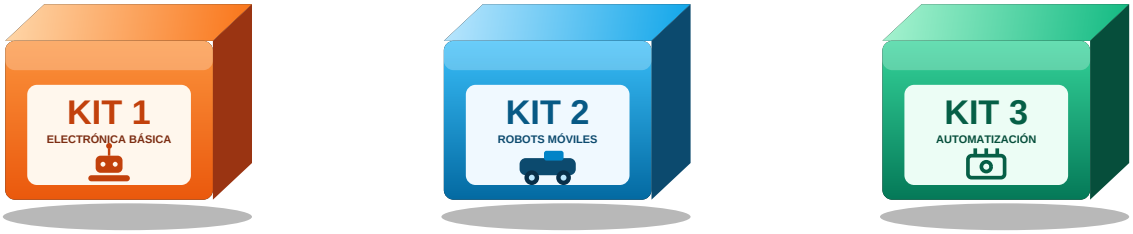
# Laboratorio Virtual de Robótica Educativa

## MANUAL DE USUARIO COMPLETO

Explorador de los 3 kits de robótica educativa + laboratorio virtual de electrónica con simulación eléctrica real, multímetro, fuente regulable e intérprete de Arduino/ESP32 con 11 prácticas guiadas.




Versión: 2026-11-02 | Sitio web: https://migulopez.github.io/kits_robotica/

Repositorio: https://github.com/migulopez/kits_robotica | Este documento: docs/MANUAL.pdf

Generado automáticamente desde los datos del proyecto (tools/export_data.js + tools/build_manual.py).

# Índice

**1. Introducción y acceso a la plataforma 4**

1.1 Acceso online . . . 4
1.2 Ejecución local . . . 4
1.3 Requisitos . . . 4

**2. El explorador de kits 5**

2.0 Cómo abrir un kit y leer las fichas . . . 5
2.1 KIT 1 — Electrónica básica y actuadores (15 componentes) . . . 5
2.2 KIT 2 — Construcción y programación de robots móviles (11 componentes) . . . 16
2.3 KIT 3 — Automatización y sistemas inteligentes (13 componentes) . . . 23

**3. El laboratorio virtual de electrónica 32**

3.1 Cómo entrar . . . 32
3.2 Partes de la pantalla . . . 32
3.3 Los 25 componentes del laboratorio (con su kit y sus pines) . . . 32
3.4 Colocar, mover, rotar y eliminar . . . 33
3.5 Tender cables entre pines . . . 33
3.6 Alimentación y medición . . . 33
3.7 Propiedades editables por componente . . . 34
3.8 Programación: editor, intérprete y monitor serial . . . 34
3.9 La pantalla LCD 16x2 con módulo I2C . . . 35
3.10 Avisos inteligentes (validación en español) . . . 35

**4. Prácticas guiadas (11 proyectos) 36**

4.1 Práctica 1: Encendido de un LED con resistencia . . . 36
4.2 Práctica 2: Control de brillo de un LED con potenciómetro (PWM) . . . 38
4.3 Práctica 3: Pantalla LCD 16x2 (I2C) mostrando «HOLA MUNDO» . . . 39
4.4 Práctica 4: LCD (I2C) mostrando el valor del potenciómetro en vivo . . . 40
4.5 Práctica 5: Tono del buzzer controlado por potenciómetro . . . 41
4.6 Práctica 6: Sensor ultrasónico HC-SR04 que mueve un servomotor . . . 42
4.7 Práctica 7: Estación climática: DHT11 + LCD I2C en ESP32 . . . 44
4.8 Práctica 8: Relé que enciende una lámpara (LED de potencia) . . . 46
4.9 Práctica 9: Servomotor controlado con un potenciómetro . . . 48
4.10 Práctica 10: Alarma de proximidad: HC-SR04 + buzzer . . . 49
4.11 Práctica 11: Luz automática nocturna con LDR . . . 51

**5. Referencia de pines de las placas y la LCD 53**

5.1 Arduino UNO . . . 53
5.2 ESP32 DevKit (CH340) . . . 53
5.3 LCD 16x2 con módulo I2C . . . 54

5.4 Multímetro digital . . . . . 54

**6. Cómo funciona la simulación eléctrica 55**

**7. Solución de problemas 55**

**8. Glosario 56**

**9. Estructura del repositorio y cómo se genera este manual 56**

# 1. Introducción y acceso a la plataforma

El Laboratorio Virtual de Robótica Educativa es una aplicación web de una sola página (HTML5 + CSS3 + JavaScript puro, sin frameworks ni dependencias externas en tiempo de ejecución) con dos módulos:

- Explorador de kits: tres cajas de kits de robótica educativa sobre una mesa de trabajo. Al hacer clic en una caja se abre el kit y se muestran todos sus componentes con ficha técnica ampliada (qué es, para qué sirve, cómo funciona, cómo se utiliza y ejemplos).
- Laboratorio virtual de electrónica: una mesa de simulación donde se colocan componentes reales, se tienden cables entre sus pines, se alimenta el circuito con fuentes regulables, se mide con un multímetro digital y se programa un Arduino UNO o un ESP32 con un intérprete controlado que ejecuta el código contra el circuito eléctrico real (resuelto con análisis nodal modificado, MNA). Incluye 11 prácticas guiadas.

## 1.1 Acceso online

La aplicación está publicada con GitHub Pages: https://migulopez.github.io/kits_robotica/

Después de cada actualización del sitio, espera 1–2 minutos a que termine el build y refresca el navegador con Ctrl+Shift+R (o abre la URL con un parámetro anti-caché, por ejemplo ?v=8). Si un botón «no aparece», casi siempre es caché del navegador.

## 1.2 Ejecución local

Descarga o clona el repositorio y abre index.html directamente en el navegador, o sirvételo con cualquier servidor estático:

```
git clone https://github.com/migulopez/kits_robotica.git
cd kits_robotica
python3 -m http.server 8000
# abre http://localhost:8000/
```

## 1.3 Requisitos

- Navegador moderno con soporte de SVG, Canvas y WebAudio (Chrome, Edge, Firefox, Safari recientes).
- Conexión a internet solo la primera vez (la app no usa librerías externas; todos los gráficos son SVG locales).
- Para ejecutar las pruebas del motor: Node.js (comando: node tests/engine.test.js).

# 2. El explorador de kits

La pantalla inicial muestra una mesa de trabajo con 3 cajas, una por kit. Encima hay un botón grande pulsante «Laboratorio virtual de electrónica» que abre directamente el laboratorio. En la parte superior también hay un filtro Todos / Kit 1 / Kit 2 / Kit 3.

## 2.0 Cómo abrir un kit y leer las fichas

• Haz clic en una caja: esa caja se «abre» y las demás se apartan; aparece la vista del kit con su título y descripción.

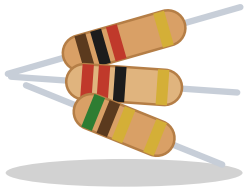
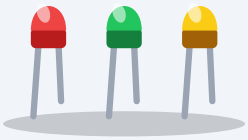
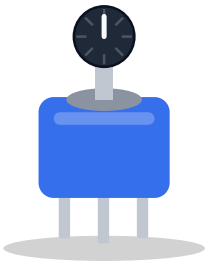
• La vista del kit muestra una rejilla de tarjetas, una por componente (imagen + nombre).

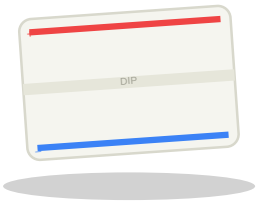
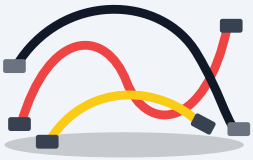
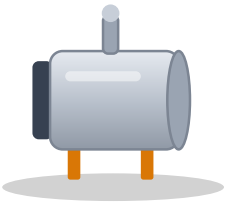
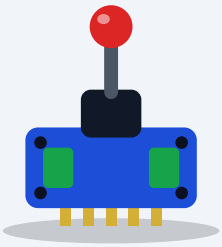
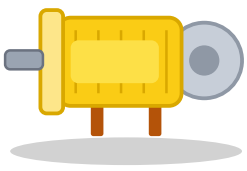
• Haz clic en una tarjeta para abrir su ficha técnica: imagen grande, qué es, para qué sirve, cómo funciona, cómo se utiliza (conexiones y código) y ejemplos. Cierra la ficha con la o haciendo clic fuera.

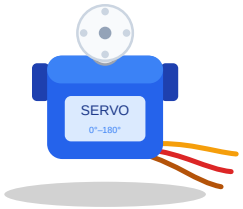
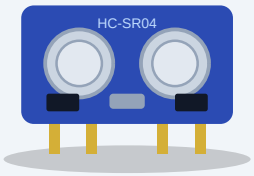
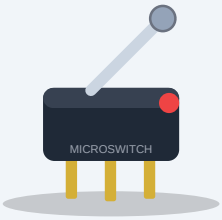
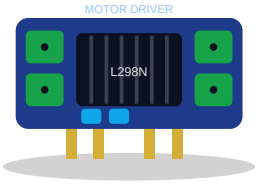
• Usa «<- Volver a la mesa» para regresar a las tres cajas, y el botón «* Laboratorio virtual» (junto a Volver) para saltar al laboratorio con ese kit preseleccionado en la paleta.

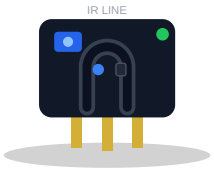
## 2.1 KIT 1 — Electrónica básica y actuadores (15 componentes)

Kit orientado al aprendizaje de electrónica básica, el control de motores y el uso de sensores con Arduino UNO.

| # | Componente | Imagen |
|---|---|---|
| 1 | Resistencias |  |
| 2 | LEDs |  |
| 3 | Potenciómetros |  |

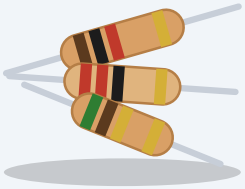
| # | Componente | Imagen |
|---|---|---|
| 4 | Protoboard |  |
| 5 | Cables jumper |  |
| 6 | Motores DC |  |
| 7 | Joystick |  |
| 8 | Motorreductor |  |

| # | Componente | Imagen |
|---|---|---|
| 9 | Servomotor |   |
| 10 | Sensor ultrasónico HC-SR04 |   |
| 11 | Sensor de aplausos LM393 |   |
| 12 | Finales de carrera (microswitch) |   |
| 13 | Driver de motores L298N |   |

| # | Componente | Imagen |
|---|---|---|
| 14 | Sensor de línea (TCRT5000) |   |
| 15 | Arduino UNO con cable USB |   |

## Fichas técnicas de los componentes del KIT 1

### Resistencias



**Qué es:** La resistencia es un componente electrónico pasivo de dos terminales que se opone al paso de la corriente eléctrica. Su valor se mide en ohmios () y en los componentes axiales se indica con bandas de colores.

**Para qué sirve:** Sirve para limitar la corriente que circula por un circuito, proteger componentes sensibles como los LEDs, crear divisores de voltaje y definir estados lógicos (pull-up / pull-down) en entradas digitales.

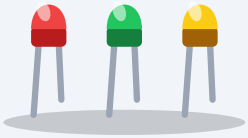
**Cómo funciona:** Se comporta según la Ley de Ohm (V = I × R): al aplicarle un voltaje, deja pasar una corriente inversamente proporcional a su resistencia y disipa el excedente de energía en forma de calor.

**Cómo se utiliza:** Se conecta directamente en el circuito (normalmente en serie con el componente a proteger) mediante la protoboard y cables jumper. No tiene polaridad: se puede conectar en cualquier sentido. Las bandas de color del cuerpo indican su valor y tolerancia.

**Ejemplos:**

- Resistencia de 220 en serie con un LED para que no se queme al conectarlo a 5 V.
- Divisor de voltaje con dos resistencias para leer el valor de un potenciómetro en una entrada analógica.
- Resistencia pull-down de 10 k en un pulsador para garantizar un nivel LOW cuando no se presiona.

## LEDs



**Qué es:** Un LED (diodo emisor de luz) es un semiconductor que emite luz cuando circula corriente eléctrica a través de él en el sentido correcto. Existen de varios colores: rojo, verde, amarillo, azul y blanco.

**Para qué sirve:** Sirve como indicador luminoso de estado, señalización, iluminación y como emisor de luz infrarroja en sensores. Es el componente de salida visual más usado en proyectos educativos.

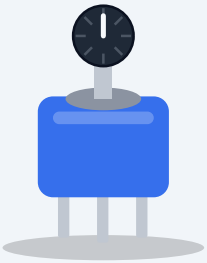
**Cómo funciona:** Al polarizarlo directamente (ánodo a positivo, cátodo a negativo) por encima de su voltaje umbral (unos 1.8–3.3 V según el color), emite luz. Si se conecta al revés no conduce ni se daña (si hay resistencia serie).

**Cómo se utiliza:** Siempre se conecta en serie con una resistencia (típicamente 220 ) entre un pin de salida del Arduino y GND. El terminal largo es el ánodo (+) y el corto (o el lado plano de la cápsula) es el cátodo (-).

**Ejemplos:**

- Semáforo con tres LEDs (rojo, amarillo, verde) controlado por un Arduino.
- Indicador de que un robot está encendido o de que una condición se cumplió.
- Linterna automática que se enciende cuando un sensor LDR detecta poca luz.

## Potenciómetros



**Qué es:** El potenciómetro es una resistencia variable de tres terminales. Al girar su eje se modifica la resistencia entre el terminal central (cursor) y los extremos, lo que permite ajustar valores de forma manual.

**Para qué sirve:** Sirve para ajustar parámetros de un circuito de forma interactiva: volumen, brillo, velocidad, sensibilidad de sensores o umbral de comparación, sin tener que cambiar componentes.

**Cómo funciona:** Funciona como un divisor de voltaje ajustable: los dos terminales extremos tienen la resistencia total fija y el cursor entrega una fracción de ese voltaje que varía entre 0 V y el voltaje de alimentación según la posición del eje.

**Cómo se utiliza:** Se conectan los extremos a VCC y GND y el cursor a una entrada analógica del Arduino (A0–A5), que leerá valores de 0 a 1023. También se usan en serie como reóstato para variar corriente.

**Ejemplos:**

- Control manual de la velocidad de un motor DC mediante un driver.
- Ajuste del umbral de detección de un sensor de línea o de aplausos.
- Control de la posición de un servomotor moviendo un mando giratorio.

## Protoboard



**Qué es:** La protoboard (placa de pruebas) es una placa con orificios interconectados internamente que permite montar circuitos electrónicos de forma rápida, sin necesidad de soldar.

**Para qué sirve:** Sirve para prototipar y probar circuitos antes de soldarlos definitivamente, permitiendo conectar, desconectar y reorganizar componentes fácilmente durante el aprendizaje.

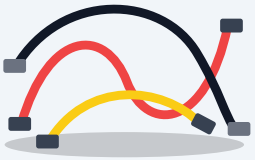
**Cómo funciona:** Internamente, las filas de 5 orificios de cada mitad están conectadas entre sí, y las columnas largos marked como + y - (líneas roja y azul) están conectadas de arriba abajo y funcionan como buses de alimentación.

**Cómo se utiliza:** Se clavan los terminales de los componentes y los cables jumper en los orificios. Respetar la separación central (el canal) para los circuitos integrados y usar las líneas +/- para distribuir 5 V y GND.

**Ejemplos:**

- Montaje de un circuito LED + resistencia + pulsador antes de soldarlo.
- Prototipo de un robot seguidor de línea con sensores y Arduino.
- Pruebas de código con motores y driver L298N sin dañar la placa final.

## Cables jumper



**Qué es:** Los cables jumper son conductores flexibles con conectores en sus extremos (macho o hembra) que sirven para interconectar componentes sobre la protoboard y la placa Arduino sin soldar.

**Para qué sirve:** Sirven para llevar señales y alimentación entre el Arduino, la protoboard y los sensores/actuadores, permitiendo montar y desmontar el circuito con facilidad.

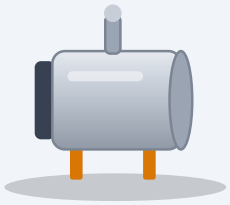
**Cómo funciona:** Son simples cables conductores: transmiten la señal eléctrica de un punto a otro. Existen tres tipos: macho-macho, macho-hembra y hembra-hembra, y de distintos colores para organizar el circuito.

**Cómo se utiliza:** Se enchufan directamente en los orificios de la protoboard o en los headers del Arduino. Conviene usar colores: rojo para VCC, negro/negro para GND y otros colores para señales.

**Ejemplos:**

- Conexión de un sensor ultrasónico a los pines Trig/Echo del Arduino.
- Alimentación de un módulo relé desde los pines 5 V y GND.
- Enlace entre la protoboard y un driver de motores con varios cables de señal.

## Motores DC



**Qué es:** El motor DC (de corriente continua) es un actuador que convierte energía eléctrica en movimiento de rotación continua mediante una bobina y un sistema de escobillas sobre un eje.

**Para qué sirve:** Sirve para generar movimiento en proyectos de robótica: ruedas de vehículos, ventiladores, cintas transportadoras o cualquier mecanismo que necesite girar continuamente.

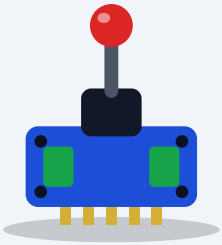
**Cómo funciona:** Al aplicarle voltaje entre sus dos terminales, la corriente circula por la bobina y genera un campo magnético que hace girar el eje. Invertir la polaridad invierte el sentido de giro; con PWM se controla la velocidad.

**Cómo se utiliza:** No se conecta directamente a un pin del Arduino (supera la corriente permitida): se usa un driver como el L298N. Se alimentan sus bornes con la salida del driver y se controlan con señales digitales y PWM.

**Ejemplos:**

- Ruedas de un carro robot controlado con un driver L298N.
- Ventilador automático que se enciende cuando el DHT11 detecta calor.
- Bomba de agua pequeña para un sistema de riego automatizado.

## Joystick



**Qué es:** El módulo joystick analógico (tipo KY-023) contiene dos potenciómetros dispuestos en ángulo recto (ejes X e Y) y un pulsador central, todo montado sobre una pequeña placa con pines.

**Para qué sirve:** Sirve como control manual proporcional: permite manejar robots, grúas o vehículos en dos direcciones y además detectar el clic del pulsador.

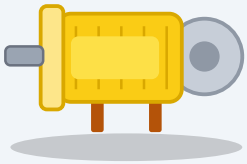
**Cómo funciona:** Al mover la palanca se desplazan los cursores de los dos potenciómetros, entregando voltajes variables entre 0 y 5 V en las salidas VRx y VRy. En reposo entrega aproximadamente 2.5 V. El pulsador entrega un contacto a GND.

**Cómo se utiliza:** Se conecta VRx y VRy a entradas analógicas del Arduino (ej. A0 y A1), SW a una entrada digital con pull-up y VCC/GND a la alimentación. Se leen con analogRead() y digitalRead().

**Ejemplos:**

- Mando de control para manejar un robot móvil en todas direcciones.
- Control de un brazo robótico de 2 grados de libertad.
- Gamepad casero para manejar un vehículo por Bluetooth.

## Motorreductor



**Qué es:** El motorreductor (tipo TT) es un motor DC pequeño unido a una caja de engranajes de plástico que reduce la velocidad de giro y aumenta la fuerza (torque) en el eje de salida.

**Para qué sirve:** Sirve cuando se necesita fuerza para mover peso a baja velocidad, como las ruedas de un robot móvil, ya que un motor DC solo no tiene suficiente torque para mover un chasis.

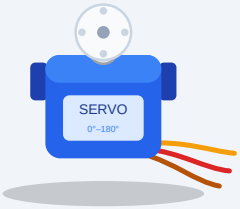
**Cómo funciona:** El motor gira muy rápido pero con poca fuerza; los engranajes de la caja reductora transmiten ese giro a un eje más lento y con mucho más torque. Invertir la polaridad invierte el sentido de giro.

**Cómo se utiliza:** Se controla igual que un motor DC, con un driver (L298N/L293D) o con transistores. Sus dos bornes se conectan a la salida del driver y la rueda se monta directamente en el eje estriado.

**Ejemplos:**

- las cuatro ruedas amarillas del kit de armado Elegoo.
- Ruedas de un robot seguidor de línea que avanza despacio y con fuerza.
- Mecanismo de apertura de una puerta automática de maqueta.

## Servomotor




**Qué es:** El servomotor es un motor con electrónica interna de control que permite posicionar su eje en un ángulo determinado, típicamente entre 0° y 180°, y mantenerlo fijo en esa posición.

**Para qué sirve:** Sirve para movimientos precisos y controlados: dirección de vehículos, brazos robóticos, barrido de sensores, apertura de compuertas, etc.

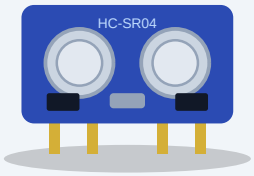
**Cómo funciona:** Recibe una señal PWM por su cable de control: la duración del pulso (aprox. 1 ms = 0°, 1.5 ms = 90°, 2 ms = 180°) indica el ángulo deseado. La electrónica interna compara la posición real con la ordenada y mueve el eje hasta alcanzarla.

**Cómo se utiliza:** Se conecta el cable marrón/rojo a GND/VCC y el de señal (amarillo/naranja) a un pin PWM del Arduino. Se controla con la librería Servo.h usando servo.write(ángulo).

**Ejemplos:**

- Barrido del sensor ultrasónico de un robot para detectar obstáculos a izquierda y derecha.
- Brazo robótico que levanta y baja objetos.
- Dirección delantera de un carro robot controlado por servos.

## Sensor ultrasónico HC-SR04




**Qué es:** El HC-SR04 es un módulo sensor de distancia por ultrasonido: emite un pulso ultrasónico y mide el tiempo que tarda el eco en regresar tras rebotar en un objeto.

**Para qué sirve:** Sirve para medir distancias sin contacto (de 2 cm a 4 m), fundamental para que los robots detecten obstáculos, midan niveles de líquidos o se estacionen solos.

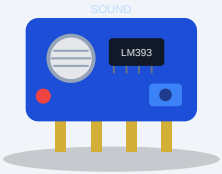
**Cómo funciona:** Un transductor emite 8 pulsos de 40 kHz; al recibir el eco el otro transductor, el módulo entrega en el pin Echo un pulso HIGH cuyo ancho es proporcional al tiempo de ida y vuelta. Distancia (cm) = tiempo (µs) / 58.

**Cómo se utiliza:** Se conectan VCC a 5 V, GND a GND, Trig y Echo a dos pines digitales del Arduino. Se envía un pulso de 10 µs por Trig y se mide la duración del pulso en Echo con pulseIn().

**Ejemplos:**

- Robot que frena o gira al detectar un obstáculo delante.
- Alarma de proximidad que avisa cuando alguien se acerca a una puerta.
- Medidor de nivel de agua en un tanque.

## Sensor de aplausos LM393




**Qué es:** Es un módulo con micrófono y un comparador LM393 que detecta sonidos fuertes como aplausos o golpes y entrega una señal digital cuando el sonido supera un umbral ajustable.

**Para qué sirve:** Sirve para crear sistemas que reaccionan al sonido: encender luces con un aplauso, activar un robot al detectar un ruido o contar palmadas.

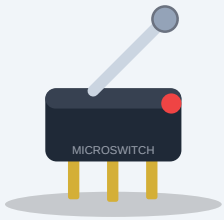
**Cómo funciona:** El micrófono convierte el sonido en una señal eléctrica; el LM393 la compara con un voltaje de referencia ajustado por el potenciómetro del módulo. Si se supera, la salida OUT pasa a LOW durante unos instantes.

**Cómo se utiliza:** Se conecta VCC a 5 V, GND a GND y OUT a una entrada digital del Arduino. El potenciómetro azul de la placa ajusta la sensibilidad: cuanto más alto el umbral, más fuerte debe ser el sonido.

**Ejemplos:**

- Lámpara que se enciende y apaga con un aplauso.
- Robot que avanza un paso cada vez que detecta una palmada.
- Sistema de alarma que se activa con un ruido fuerte en la noche.

## Finales de carrera (microswitch)




**Qué es:** El final de carrera es un microswitch: un interruptor eléctrico con una palanca metálica que se acciona mecánicamente al ser presionada, abriendo o cerrando un contacto.

**Para qué sirve:** Sirve para detectar contacto físico o llegada a un límite: choques de un robot, tope de un eje móvil, apertura de puertas o botones de emergencia.

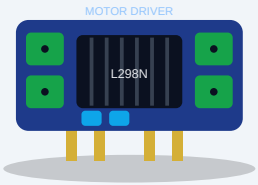
**Cómo funciona:** Al presionar la palanca, un contacto interno cambia de estado (de NC a NA o viceversa). Hay tres terminales: COM (común), NA (normalmente abierto) y NC (normalmente cerrado).

**Cómo se utiliza:** Se conecta COM a GND y NA (o NC) a una entrada digital del Arduino configurada con resistencia pull-up interna (pinMode(pin, INPUT_PULLUP)), de modo que al presionar la palanca la entrada lea LOW.

**Ejemplos:**

- Parachoques de un robot: al chocar, invierte la dirección automáticamente.
- Límite superior de un ascensor de maqueta que detiene el motor.
- Botón de emergencia que detiene todo el sistema.

## Driver de motores L298N




**Qué es:** El módulo L298N es un driver basado en el circuito integrado puente H doble L298, montado en una placa con disipador, bornes y reguladores, capaz de controlar dos motores DC o un motor paso a paso.

**Para qué sirve:** Sirve para controlar la velocidad y el sentido de giro de motores DC con las señales débiles de un microcontrolador, ya que un pin de Arduino no puede alimentar un motor directamente.

**Cómo funciona:** Cada puente H usa transistores que conectan el motor a +V o a GND en cualquier combinación: con IN1=alto e IN2=bajo el motor gira en un sentido; al revés, gira al otro; con ENA/ENB en PWM se regula la velocidad.

**Cómo se utiliza:** Se alimenta con 7–12 V en el borne de entrada (el módulo tiene un regulador a 5 V que puede alimentar el Arduino con el jumper puesto) y los motores en los bornes de salida. Los pines IN1–IN4 y ENA/ENB van a pines digitales/PWM del Arduino.

**Ejemplos:**

- Control de las dos ruedas de un robot móvil (adelante, atrás, giros).
- Control de velocidad de un motor con PWM en el pin ENA.
- Manejo de un motor paso a paso bipolar en un brazo robótico.

## Sensor de línea (TCRT5000)




**Qué es:** El sensor de línea es un módulo con un LED infrarrojo emisor y un fototransistor receptor (TCRT5000) que detecta diferencias de reflectividad entre superficies claras y oscuras.

**Para qué sirve:** Sirve para que un robot siga una línea negra sobre fondo blanco (o viceversa) detectando en qué lado del sensor está la línea.

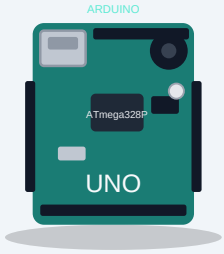
**Cómo funciona:** El LED IR ilumina la superficie; si esta es clara (blanca) refleja mucha luz y el fototransistor conduce, dando salida baja; si es oscura (línea negra) refleja poca luz y la salida queda alta.

**Cómo se utiliza:** Se conecta VCC a 5 V, GND a GND y OUT a una entrada digital del Arduino. El potenciómetro del módulo ajusta la sensibilidad. Se monta en la parte frontal del robot apuntando al suelo.

**Ejemplos:**

- Robot seguidor de línea negra con 1, 2 o 3 sensores.
- Conteo de objetos que pasan por una banda transportadora.
- Detección de bordes de una mesa para que el robot no se caiga.

## Arduino UNO con cable USB




**Qué es:** El Arduino UNO es una placa de desarrollo con el microcontrolador ATmega328P: 14 pines digitales (6 con PWM), 6 entradas analógicas, 16 MHz, mémoire flash de 32 KB y conexión USB para programarla y alimentarla.

**Para qué sirve:** Es el cerebro de los proyectos: lee sensores, ejecuta el programa y controla actuadores (motores, servos, LEDs, pantallas, relés).

**Cómo funciona:** Ejecuta en bucle las instrucciones del programa cargado (setup y loop): lee el estado de sus entradas, procesa los datos con su CPU y actualiza las salidas. Se programa en C++ con el Arduino IDE.

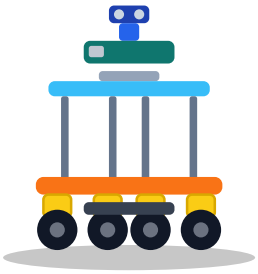
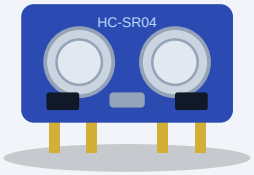
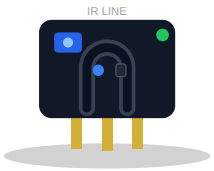
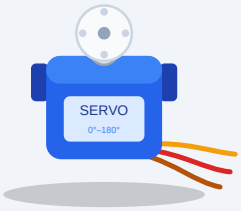
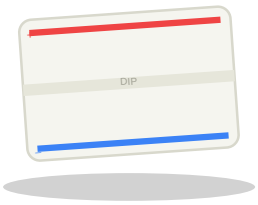
**Cómo se utiliza:** Se conecta al computador con el cable USB, que alimenta la placa (5 V) y permite subir el código. Los sensores y actuadores se conectan a sus pines mediante cables jumper y la protoboard.

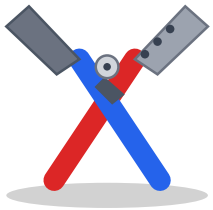
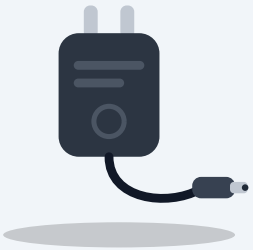
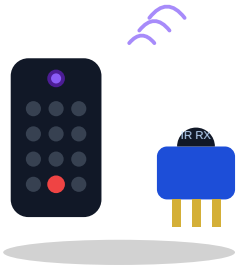
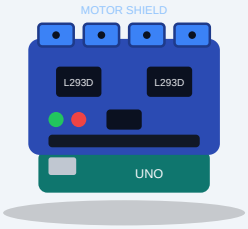
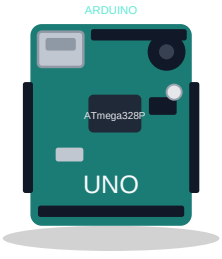
**Ejemplos:**

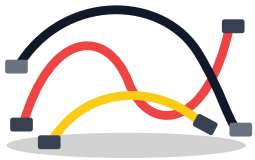
- Controlador principal de los tres kits de esta plataforma.
- Robot seguidor de línea con sensores IR y driver L298N.
- Estación meteorológica con DHT11 que muestra datos en una pantalla LCD.

## 2.2 KIT 2 — Construcción y programación de robots móviles (11 componentes)

Kit orientado al armado de robots móviles, el control de motores y la integración de sensores para crear vehículos autónomos.

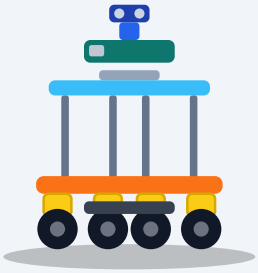
| # | Componente | Imagen |
|---|---|---|
| 1 | Kit de armado de robot Elegoo |  |
| 2 | Sensor ultrasónico |  |
| 3 | Sensor de línea infrarrojo |  |
| 4 | Servomotor azul |  |
| 5 | Protoboard |  |

| # | Componente | Imagen |
|---|---|---|
| 6 | Pinzas para pelar cables |  |
| 7 | Adaptador de corriente |  |
| 8 | Sensor infrarrojo con control remoto |  |
| 9 | Shield de control de motores L293D |  |
| 10 | Arduino UNO |  |

| # | Componente | Imagen |
|---|---|---|
| 11 | Cables jumper |  |

## Fichas técnicas de los componentes del KIT 2

### Kit de armado de robot Elegoo



**Qué es:** Es el conjunto completo para armar un robot móvil tipo 'smart car': chasis de acrílico con dos niveles, cuatro motorreductores amarillos con ruedas, soportes, tornillería, portapilas y la placa de control.

**Para qué sirve:** Sirve como base mecánica y estructural de un vehículo robot: sobre él se montan los sensores, la electrónica de control y la alimentación para construir un robot funcional.

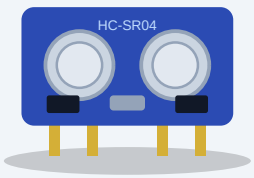
**Cómo funciona:** Los cuatro motorreductores amarillos mueven las ruedas de forma independiente; combinando sus sentidos de giro el vehículo avanza, retrocede y gira. El chasis de dos niveles separa la electrónica (arriba) de baterías y motores (abajo).

**Cómo se utiliza:** Se arma siguiendo el manual del fabricante: se atornillan los motores al chasis inferior, se monta la placa superior con separadores, se instala el Arduino y el shield de motores, y se cablean motores y sensores con los jumpers.

**Ejemplos:**

- Carro robot seguidor de línea negra usando los sensores IR frontales.
- Vehículo que esquiva obstáculos con el sensor ultrasónico montado en un servo.
- Auto controlado a distancia por Bluetooth o por control infrarrojo.

### Sensor ultrasónico




**Qué es:** Módulo HC-SR04 de medición de distancia por ultrasonido (2 cm a 4 m), con dos transductores cilíndricos sobre una placa azul.

**Para qué sirve:** En el robot móvil sirve para detectar obstáculos delante y tomar decisiones: frenar, girar o rodear el objeto.

**Cómo funciona:** Emite un tren de pulsos de 40 kHz y mide el tiempo del eco; distancia = tiempo × velocidad del sonido / 2. En el código: distancia (cm) = duración del pulso en µs / 58.

**Cómo se utiliza:** VCC a 5 V, GND a GND, Trig y Echo a pines digitales del Arduino. Montado en la parte frontal del chasis (a menudo sobre un servo para barrer).

**Ejemplos:**

- Modo 'esquiva obstáculos' del Elegoo: si distancia < 20 cm, girar.
- Robot que se estaciona solo midiendo el espacio disponible.
- Alarma de proximidad en un vehículo de vigilancia.

## Sensor de línea infrarrojo




**Qué es:** Módulo IR con LED emisor infrarrojo y fototransistor receptor que distingue superficies claras de oscuras según la luz reflejada.

**Para qué sirve:** En el robot móvil detecta la línea negra del circuito para que el vehículo la siga automáticamente sin intervención humana.

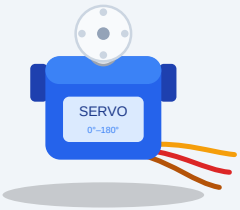
**Cómo funciona:** Sobre fondo blanco la luz IR se refleja y la salida del módulo queda en un nivel; sobre la línea negra no hay reflexión y la salida cambia. Comparando varios sensores se sabe si el robot se desvía a izquierda o derecha.

**Cómo se utiliza:** VCC a 5 V, GND a GND y OUT a una entrada digital. Se fija en la parte inferior frontal del chasis, a pocos milímetros del suelo, con el potenciómetro ajustado para distinguir bien blanco y negro.

**Ejemplos:**

- Seguidor de línea con 2 sensores: corrige la trayectoria comparando ambos.
- Detección del borde de la mesa para evitar caídas.
- Parada automática al llegar a una marca negra de 'fin de pista'.

## Servomotor azul




**Qué es:** Servomotor de carcasa azul (formato estándar tipo SG90) con eje de salida, brazo (horn) de plástico y cable de tres hilos.

**Para qué sirve:** En el robot móvil se usa para mover el sensor ultrasónico de izquierda a derecha y 'mirar' alrededor, o para dirigir ruedas delanteras.

**Cómo funciona:** La señal PWM indica el ángulo (pulso de ~1 ms a ~2 ms para 0°–180°). La electrónica interna posiciona y mantiene el eje en ese ángulo con fuerza suficiente para mover cargas ligeras.

**Cómo se utiliza:** Cable marrón a GND, rojo a 5 V y amarillo/naranja a un pin PWM del Arduino o del shield. Se programa con la librería Servo: servo.write(90) lo centra, por ejemplo.

**Ejemplos:**

- Soporte oscilante del HC-SR04 para detectar obstáculos en 180°.
- Dirección de un triciclo robot con rueda delantera motriz.
- Barrera automática que sube y baja el brazo al detectar presencia.

## Protoboard



**Qué es:** Placa de pruebas sin soldadura con orificios interconectados en filas de 5 y buses de alimentación (+) y (-) a los lados.

**Para qué sirve:** Permite interconectar sensores, módulos y el Arduino durante el armado del robot sin soldar, facilitando cambios y pruebas.

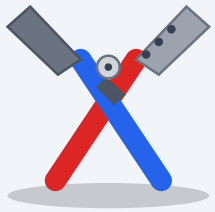
**Cómo funciona:** Cada grupo de 5 orificios está conectado internamente; las columnas roja (+) y azul (-) recorren toda la placa como buses de VCC y GND.

**Cómo se utiliza:** Se enchufan los componentes y jumpers respetando el canal central para los circuitos integrados; se usan las líneas de alimentación para repartir 5 V y GND a varios módulos.

**Ejemplos:**

- Distribución de alimentación a varios sensores del robot Elegoo.
- Conexión de pulsadores y LEDs indicadores en el tablero de control.
- Pruebas rápidas de motores antes del montaje definitivo.

## Pinzas para pelar cables



**Qué es:** Herramienta manual tipo pelacables/stripper: dos mangos aislados que pivotan sobre un tornillo y una cabeza metálica con orificios calibrados y filo de corte.

**Para qué sirve:** Sirve para retirar el aislante plástico de los extremos de los cables sin dañar los hilos de cobre internos, dejándolos listos para enchufar en la protoboard.

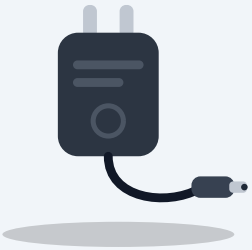
**Cómo funciona:** Al apretar los mangos, los orificios calibrados de la cabeza cierran sobre el aislante y lo cortan; tirando del cable se desprende el plástico sin cortar el cobre. El filo superior sirve para cortar el cable a medida.

**Cómo se utiliza:** Se elige el orificio del diámetro adecuado al cable, se inserta unos 5–10 mm de punta, se aprieta y se tira. Siempre con la herramienta, no con los dientes, para no dañar el conductor.

**Ejemplos:**

- Preparar los extremos de los cables de los motores del Elegoo.
- Hacer jumpers caseros a la medida necesaria.
- Reparar cables de alimentación pelados o dañados.

## Adaptador de corriente



**Qué es:** Fuente de alimentación tipo 'wall wart': un adaptador AC-DC con clavija de pared y cable con conector cilíndrico (barrel jack) de 5.5 × 2.1 mm, típicamente de 9 V o 12 V.

**Para qué sirve:** Sirve para alimentar el robot y los motores desde la red eléctrica cuando las pilas no son suficientes, usando el conector de alimentación (jack) del Arduino o del driver.

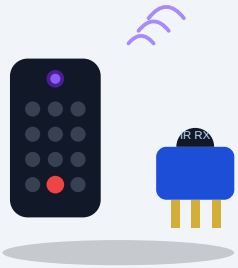
**Cómo funciona:** Transforma la corriente alterna de la pared (110/220 V) en corriente continua estable de bajo voltaje (ej. 9 V DC) con la corriente máxima indicada en su etiqueta (ej. 1 A).

**Cómo se utiliza:** Se enchufa a la corriente y su plug cilíndrico entra en el jack de alimentación del Arduino UNO o en el borne de entrada del driver L298N. Respetar la polaridad (centro positivo) y no exceder el voltaje máximo.

**Ejemplos:**

- Alimentación externa del robot Elegoo para pruebas prolongadas.
- Fuente estable para motores que consumen más que lo que entrega el USB.
- Proyecto fijo de escritorio que no depende de pilas.

## Sensor infrarrojo con control remoto



**Qué es:** Conjunto de dos partes: un control remoto con LED infrarrojo que emite códigos al presionar botones, y un receptor IR (tipo VS1838B) con cúpula oscura que recibe esos códigos.

**Para qué sirve:** Sirve para manejar el robot a distancia: avanzar, girar, detenerse o ejecutar modos predefinidos desde el control remoto.

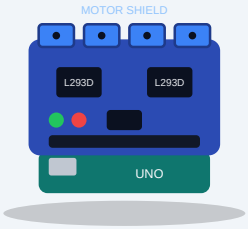
**Cómo funciona:** El control emite pulsos de luz infrarroja (940 nm) codificados según el protocolo NEC (cada botón tiene un código único). El receptor demodula la señal y entrega pulsos digitales que el Arduino decodifica con la librería IRremote.

**Cómo se utiliza:** El receptor tiene 3 pines: VCC a 5 V, GND a GND y OUT a un pin digital del Arduino. Se usa la librería IRremote: irrecv.decode() entrega el código del botón presionado y se ejecuta la acción correspondiente.

**Ejemplos:**

- Robot Elegoo manejado con las flechas del control remoto.
- Selección de modos (seguidor de línea, esquiva obstáculos) con botones numéricos.
- Brazo robótico movido a distancia con un mando IR.

## Shield de control de motores L293D




**Qué es:** Es una placa (shield) que se encaja directamente sobre el Arduino UNO y lleva dos circuitos L293D de doble puente H, bornes azules para motores y conectores para servos.

**Para qué sirve:** Sirve para controlar hasta 4 motores DC, 2 servos o 1 motor paso a paso sin cableado suelto: al apilarla sobre el Arduino, los motores se conectan directamente en sus bornes.

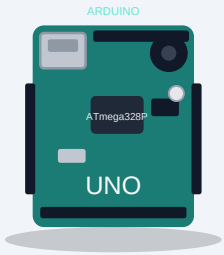
**Cómo funciona:** Cada L293D contiene dos puentes H que permiten invertir la polaridad de cada motor. Las señales se enrutan internamente a pines fijos del Arduino (ej. pines 3, 5, 6, 9, 10, 11 en la versión clásica) controlados con la librería AFMotor.

**Cómo se utiliza:** Se apila sobre el Arduino UNO (pin a pin) y los cables de cada motor se atornillan en los bornes M1–M4. Se programa con la librería Adafruit Motor Shield: motor.setSpeed(255) y motor.run(FORWARD).

**Ejemplos:**

- Control de los 4 motores del Elegoo con un solo shield apilado.
- Manejo de 2 servos para una torreta con sensor ultrasónico.
- Robot de 2 ruedas con control simple de velocidad y dirección.

## Arduino UNO




**Qué es:** Placa Arduino UNO con microcontrolador ATmega328P: 14 pines digitales (6 PWM), 6 entradas analógicas, USB y jack de alimentación. Cerebro del robot móvil.

**Para qué sirve:** Controla todo el vehículo: lee los sensores (ultrasónico, línea, IR) y envía órdenes a los motores a través del shield o driver.

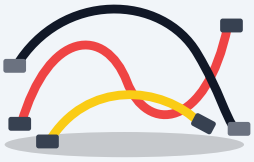
**Cómo funciona:** Ejecuta el programa cargado por USB: en el loop() lee sensores, toma decisiones (if/else) y ordena a los motores avanzar, girar o detenerse.

**Cómo se utiliza:** Se monta sobre el chasis del robot (o debajo del shield), se alimenta por USB o por el jack con el adaptador, y se programa desde el Arduino IDE.

**Ejemplos:**

- Programa 'esquiva obstáculos' del Elegoo con HC-SR04 y 4 motores.
- Seguidor de línea con corrección proporcional de dirección.
- Control por infrarrojo usando el receptor y el control remoto.

## Cables jumper



**Qué es:** Cables conductores con conectores macho/hembra en los extremos, de varios colores, para interconectar la electrónica del robot sin soldar.

**Para qué sirve:** Sirven para unir sensores, módulos y motores con el Arduino/shield, permitiendo montar y desmontar el circuito fácilmente.

**Cómo funciona:** Conducen la señal o la alimentación de un punto a otro. Los de color rojo suelen usarse para VCC, negro para GND y otros colores para señales.

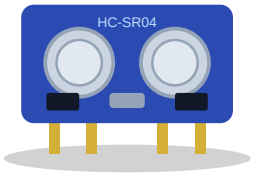
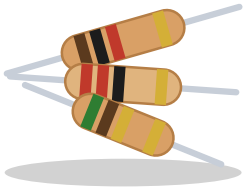
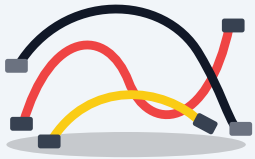
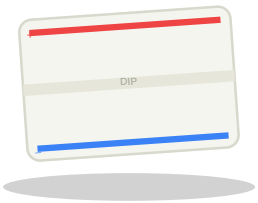
**Cómo se utiliza:** Se enchufan en los headers del shield/Arduino y en los pines de los módulos. Conviene ordenarlos por color para encontrar fallos rápido.

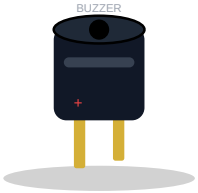
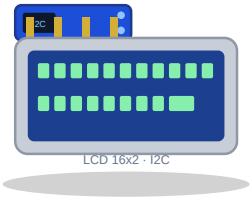
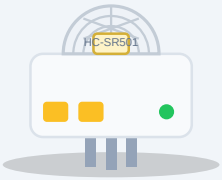
**Ejemplos:**

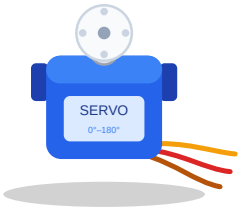
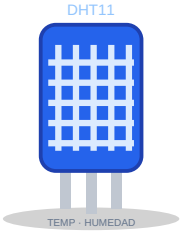
- Enlace de los sensores IR del chasis con el shield L293D.
- Conexión del receptor IR y del servo de la torreta.
- Alimentación de la protoboard desde los pines 5 V y GND.

## 2.3 KIT 3 — Automatización y sistemas inteligentes (13 componentes)

Kit orientado al desarrollo de sistemas de automatización, monitoreo ambiental y control con microcontroladores y conectividad (ESP32).

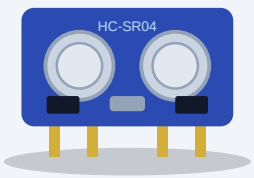
| # | Componente | Imagen |
|---|---|---|
| 1 | Sensor ultrasónico |   |
| 2 | Módulo de aplausos |   |
| 3 | Resistencias |  |
| 4 | Cables jumper |  |
| 5 | Protoboard |   |

| # | Componente | Imagen |
|---|---|---|
| 6 | Buzzer |   |
| 7 | Módulo relé de un canal |   |
| 8 | Pantalla LCD 16x2 con módulo I2C |   |
| 9 | Sensor de presencia HC-SR501 (PIR) |   |
| 10 | Módulo LDR (sensor de luz) |   |

| # | Componente | Imagen |
|---|---|---|
| 11 | Servomotor |   |
| 12 | ESP32 DevKit con CH340 |   |
| 13 | Sensor DHT11 |   |

## Fichas técnicas de los componentes del KIT 3

### Sensor ultrasónico




**Qué es:** Módulo HC-SR04 que mide distancia por eco de ultrasonido, entre 2 cm y 4 m, con pines Trig y Echo.

**Para qué sirve:** En automatización sirve para detectar presencia o proximidad: abrir una puerta automática, contar objetos o activar una alarma cuando algo se acerca.

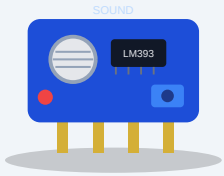
**Cómo funciona:** Emite 8 pulsos de 40 kHz y mide el tiempo del eco en el pin Echo; distancia (cm) = tiempo (µs) / 58.

**Cómo se utiliza:** VCC a 5 V, GND a GND, Trig y Echo a pines digitales del ESP32/Arduino. En ESP32 se usa pulseIn() igual que en Arduino.

**Ejemplos:**

- Puerta automática de maqueta que se abre al detectar una mano.
- Alarma de estacionamiento para un vehículo.
- Dispensador que detecta la presencia de un vaso.

## Módulo de aplausos




**Qué es:** Módulo con micrófono y comparador LM393 que entrega una señal digital cuando detecta un sonido fuerte (aplauso, grito, golpe).

**Para qué sirve:** Sirve para activar acciones con la voz o el sonido: encender luces, contar palmadas o disparar una alarma por ruido.

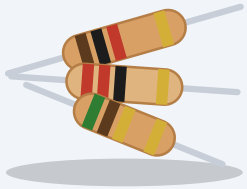
**Cómo funciona:** El micrófono genera una señal proporcional al sonido; el LM393 la compara con el umbral del potenciómetro y conmuta la salida OUT (activa en bajo) durante unos cientos de milisegundos.

**Cómo se utiliza:** VCC a 5 V (o 3.3 V en ESP32), GND a GND y OUT a una entrada digital. Ajustar el trimmer azul hasta que el LED indicador solo se encienda con sonidos fuertes.

**Ejemplos:**

- Luz de la habitación que se controla con aplausos.
- Sistema de 'aplausómetro' que cuenta cuántas veces aplaude el público.
- Alarma antirrobo que suena ante ruidos fuertes.

## Resistencias



**Qué es:** Componentes pasivos que limitan el paso de la corriente eléctrica; su valor en ohmios se lee por las bandas de colores.

**Para qué sirve:** Protegen LEDs y sensores, crean divisores de voltaje para leer el LDR o el potenciómetro, y fijan estados pull-up/pull-down en entradas digitales.

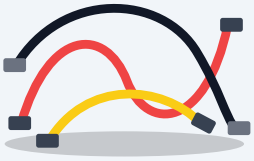
**Cómo funciona:** Obedecen la Ley de Ohm (V = I × R): a mayor voltaje, la corriente que dejan pasar es menor según su resistencia; el excedente se disipa como calor.

**Cómo se utiliza:** Se conectan en serie con lo que se quiere proteger (ej. LED + 220 ) o como divisor con el LDR. No tienen polaridad.

**Ejemplos:**

- LED indicador de estado del sistema con su resistencia de 220 .
- Divisor de voltaje con el módulo LDR para medir la luz ambiental.
- Pull-down en pulsadores de control del sistema domótico.

## Cables jumper



**Qué es:** Cables con conectores en los extremos para interconectar la protoboard, el ESP32 y los módulos sin soldar.

**Para qué sirve:** Organizan el cableado del sistema de automatización: alimentación y señales entre sensores, actuadores y la placa.

**Cómo funciona:** Son conductores que llevan señal o alimentación de un punto a otro; el color ayuda a identificar VCC, GND y señales.

**Cómo se utiliza:** Se enchufan en los pines GPIO del ESP32 (con cuidado: algunos pines son solo entrada) y en la protoboard.

**Ejemplos:**

- Conexión del DHT11, LCD y relé al ESP32.
- Alimentación de la protoboard desde 3.3 V y GND del ESP32.
- Enlace de sensores ambientales en una estación meteorológica.

## Protoboard




**Qué es:** Placa de pruebas sin soldadura con filas de 5 orificios conectados y buses de + y - a los lados.

**Para qué sirve:** Permite montar el circuito del sistema de automatización (sensores + actuadores) y probarlo antes de la instalación definitiva.

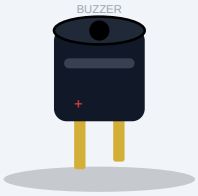
**Cómo funciona:** Las filas de 5 orificios están unidas internamente; las columnas roja/azul son buses de alimentación que recorren toda la placa.

**Cómo se utiliza:** Se enchufan componentes y jumpers; se usa el canal central para los integrados y los buses para VCC/GND. En ESP32, alimentar con 3.3 V.

**Ejemplos:**

- Montaje del circuito del relé con su transistor y LED indicador.
- Pruebas del buzzer y del módulo de aplausos.
- Prototipo de la estación ambiental con DHT11 y LCD.

## Buzzer




**Qué es:** El buzzer es un transductor electroacústico cilíndrico que convierte una señal eléctrica en sonido; puede ser activo (tono fijo) o pasivo (reproduce frecuencias).

**Para qué sirve:** Sirve para dar avisos sonoros: alarmas, confirmación de acciones, melodías de aviso o alertas de sensores en el sistema de automatización.

**Cómo funciona:** Al pasar corriente por su bobina, un diafragma vibra produciendo sonido. El buzzer activo tiene oscilador interno (basta darle DC); el pasivo necesita una señal cuadrada de frecuencia definida (tone() en Arduino).

**Cómo se utiliza:** Pin positivo (+) al pin digital del microcontrolador (con resistencia si es necesario) y pin negativo a GND. Activo: digitalWrite HIGH suena. Pasivo: tone(pin, frecuencia, duración).

**Ejemplos:**

- Alarma que suena cuando el HC-SR501 detecta movimiento.
- Aviso de que la puerta automática se está abriendo.
- Melodía de confirmación al registrar la huella/tecla correcta.

## Módulo relé de un canal




**Qué es:** Módulo con un relé electromecánico (ej. SRD-05VDC) montado en placa azul, con bornes de tornillo (COM, NC, NO), optoacoplador y LED indicador.

**Para qué sirve:** Sirve como interruptor de potencia: permite que el microcontrolador encienda y apague appareils de alto voltaje (lámparas, bombas, electrodomésticos a 110/220 V) con total aislamiento eléctrico.

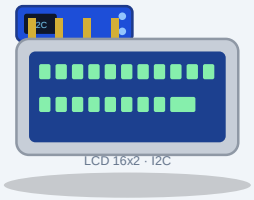
**Cómo funciona:** Al aplicar 5 V (o 3.3 V según el módulo) en IN, la bobina del relé se energiza y el contacto móvil salta de NC a NO, cerrando el circuito de la carga. Al quitar la señal, vuelve a NC.

**Cómo se utiliza:** La señal IN va a un pin digital del ESP32/Arduino; VCC y GND a la alimentación. La carga se conecta en serie: fase de la red -> borne COM del relé -> borne NO -> aparato -> neutro. ¡Trabajar con precaución en 110/220 V!

**Ejemplos:**

- Encender una lámpara de la casa desde una app con el ESP32.
- Activar una bomba de riego automático según la humedad del DHT11.
- Sistema de alarma que enciende una sirena de 110 V.

## Pantalla LCD 16x2 con módulo I2C




**Qué es:** Pantalla de cristal líquido de 16 columnas × 2 filas con retroiluminación azul y controlador HD44780, que en este kit viene con el módulo I2C (backpack PCF8574) ya soldado detrás.

**Para qué sirve:** Sirve para mostrar información del sistema al usuario: lecturas de temperatura, mensajes de estado, menús o alertas, sin necesidad de computador.

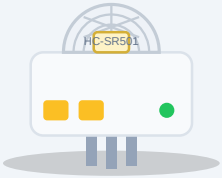
**Cómo funciona:** El módulo I2C traduce el protocolo serie I2C (pines SDA y SCL, dirección habitual 0x27) a las señales del controlador HD44780, así que la pantalla solo ocupa 2 pines del microcontrolador más la alimentación.

**Cómo se utiliza:** VCC a 5 V, GND a GND, SDA a GPIO21 (ESP32) o A4 (UNO), SCL a GPIO22 (ESP32) o A5 (UNO). Se programa con la librería LiquidCrystal_I2C: LiquidCrystal_I2C lcd(0x27, 16, 2); lcd.setCursor(0,0); lcd.print("Hola").

**Ejemplos:**

- Estación meteorológica que muestra temperatura y humedad del DHT11.
- Menú del sistema domótico para elegir el dispositivo a controlar.
- Mensaje 'Puerta abierta' cuando el ultrasónico detecta presencia.

## Sensor de presencia HC-SR501 (PIR)




**Qué es:** Sensor de movimiento por infrarrojos pasivos (PIR) con cúpula blanca de Fresnel: detecta cambios en la radiación infrarroja cuando una persona u objeto caliente cruza su campo de visión.

**Para qué sirve:** Sirve para detectar presencia y movimiento: alarmas, luces automáticas, sistemas de vigilancia o activación de dispositivos al pasar alguien.

**Cómo funciona:** Dos elementos piroeléctricos comparan la radiación IR recibida; al moverse una fuente de calor (cuerpo humano) entre zonas, la diferencia supera el umbral y la salida OUT pasa a HIGH durante un tiempo ajustable (2 s a varios minutos).

**Cómo se utiliza:** VCC a 5 V (acepta 4.5–20 V), GND a GND y OUT a un pin digital. Los dos potenciómetros ajustan el tiempo de salida y la sensibilidad/distancia (hasta ~7 m).

**Ejemplos:**

- Luz automática del pasillo que se enciende al detectar movimiento.
- Alarma de seguridad con buzzer y aviso al celular vía ESP32.
- Contador de personas que entran a un salón.

## Módulo LDR (sensor de luz)




**Qué es:** Módulo con una fotorresistencia LDR: un componente cuya resistencia eléctrica disminuye al aumentar la luz que recibe, montado en placa con comparador LM393.

**Para qué sirve:** Sirve para medir la intensidad luminosa del ambiente: encender luces automáticamente de noche, ajustar brillo de pantallas o registrar datos de iluminación.

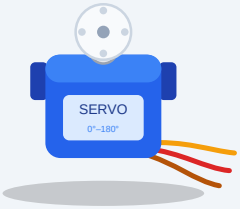
**Cómo funciona:** La LDR está hecha de un semiconductor (sulfuro de cadmio) cuya resistencia baja de megaohmios en la oscuridad a pocos cientos de ohmios a plena luz. En un divisor de voltaje con una resistencia fija, el voltaje resultante varía con la luz.

**Cómo se utiliza:** El módulo puede dar salida digital (DO, ajustada con el potenciómetro) o analógica (AO) a una entrada analógica del microcontrolador, que lee valores de 0 a 1023 (UNO) o 0 a 4095 (ESP32).

**Ejemplos:**

- Lámpara automática que se enciende cuando oscurece.
- Seguidor solar: un servo orienta un panel hacia la luz.
- Registro de horas de sol en una estación meteorológica con ESP32.

## Servomotor




**Qué es:** Servomotor de posición controlada (0°–180°) con eje, brazo plástico y cable de 3 hilos.

**Para qué sirve:** En automatización mueve compuertas, válvulas, persianas o mecanismos de apertura controlada desde el sistema inteligente.

**Cómo funciona:** La duración del pulso PWM en su cable de señal determina el ángulo del eje (1 ms -> 0°, 1.5 ms -> 90°, 2 ms -> 180°), que el servo mantiene con torque.

**Cómo se utiliza:** Alimentación 5 V (rojo/marrón) y señal PWM (amarillo/naranja) a un pin del ESP32. En ESP32 se usa la librería ESP32Servo: servo.write(ángulo).

**Ejemplos:**

- Apertura automática de una puerta con el HC-SR501.
- Válvula de riego controlada por la humedad del suelo.
- Persiana que se orienta según la luz medida por el LDR.

## ESP32 DevKit con CH340




**Qué es:** Placa de desarrollo ESP32 DevKit: microcontrolador de doble núcleo a 240 MHz con WiFi y Bluetooth integrados, 30+ pines GPIO y puerto USB con chip CH340.

**Para qué sirve:** Es el cerebro de los sistemas inteligentes conectados: además de leer sensores y mover actuadores, publica los datos en internet y recibe órdenes desde una app o página web.

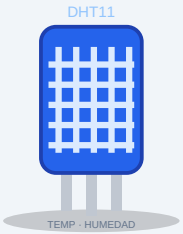
**Cómo funciona:** Ejecuta el programa en su memoria flash y se conecta a redes WiFi como cliente o crea su propio punto de acceso. Los pines GPIO admiten entradas/salidas digitales, analógicas (ADC), PWM (LEDC), I2C, SPI y UART.

**Cómo se utiliza:** Se programa con Arduino IDE (paquete ESP32) o MicroPython conectando el USB (driver CH340). Se alimenta por USB (5 V) o por el pin Vin (7–12 V). Cuidado: sus pines trabajan a 3.3 V.

**Ejemplos:**

- Sistema domótico: controla relés y lee sensores desde el celular por WiFi.
- Estación meteorológica con DHT11 que publica datos en ThingSpeak.
- Robot móvil con cámara ESP32-CAM controlado por una página web.

## Sensor DHT11




**Qué es:** Sensor digital de temperatura y humedad relativa en carcasa azul con rejilla de ventilación: mide de 0 a 50 °C (±2 °C) y 20–90 % de humedad (±5 %).

**Para qué sirve:** Sirve para el monitoreo ambiental: clima de una habitación, invernaderos, secado de alimentos o condiciones de almacenamiento, enviando los datos a una pantalla o a internet.

**Cómo funciona:** Usa un termistor y un sensor de humedad capacitivo; un microcontrolador interno mide ambos y envía el resultado como señal digital serie por un único pin de datos (protocolo propio de 40 bits).

**Cómo se utiliza:** Tiene 3 pines: VCC a 3.3–5 V, GND a GND y DATA al pin digital del microcontrolador con una resistencia pull-up de 4.7–10 k. Se lee con la librería DHT: dht.readTemperature() y dht.readHumidity().

**Ejemplos:**

- Invernadero automatizado: si temperatura > 30 °C, enciende un ventilador (relé).
- Alerta de humedad alta en un cuarto de servidores.
- Estación meteorológica con LCD 16x2 y publicación web en ESP32.

# 3. El laboratorio virtual de electrónica

## 3.1 Cómo entrar

• Desde la pantalla de inicio: botón grande «* Laboratorio virtual de electrónica» del encabezado.

• Desde la vista de un kit: botón «* Laboratorio virtual» junto a «<- Volver a la mesa» (la paleta queda filtrada a ese kit).

• Para salir: «<- Volver a los kits».

## 3.2 Partes de la pantalla

| Zona | Qué contiene | Para qué sirve |
|---|---|---|
| Barra superior | <- Volver - [ON/OFF] Alimentación ON/OFF - Reiniciar - Limpiar mesa - Prácticas guiadas - zoom - 100% + - Rotar componente | Control general del laboratorio |
| Paleta izquierda | Filtro de kit (Todos/Kit 1/2/3) - selector de color de cable - 9 categorías con los 25 componentes y su etiqueta de kit (K1-K2…) | Arrastrar componentes a la mesa |
| Mesa central | Mundo virtual de 4000×4000 px con rejilla, componentes y capa SVG de cables | Montar y cablear el circuito |
| Panel derecho | Propiedades del componente seleccionado Avisos del circuito Editor de código + > Cargar y ejecutar / [] Detener + monitor serial | Configurar, validar y programar |

## 3.3 Los 25 componentes del laboratorio (con su kit y sus pines)

| Componente | Categoría | Kit | Pines (en el orden del dibujo) |
|---|---|---|---|
| Fuente regulable 0–12 V | Alimentación | Kit 1, Kit 2, Kit 3 | + (VCC), - (GND) |
| Pila 9 V | Alimentación | Kit 1, Kit 2, Kit 3 | + (VCC), - (GND) |
| Adaptador de corriente 12 V | Alimentación | Kit 2 | + (centro) (VCC), - (GND) |
| Resistencia | Componentes pasivos | Kit 1, Kit 3 | Terminal A, Terminal B |
| Potenciómetro 10 k | Componentes pasivos | Kit 1 | A, Cursor W, B |
| LED | Indicadores | Kit 1 | Ánodo (+) (VCC), Cátodo (-) (GND) |
| Buzzer | Indicadores | Kit 3 | + (VCC), - (GND) |
| Arduino UNO | Controladores | Kit 1, Kit 2 | D0, D1, D2, D3~, D4, D5~, D6~, D7, D8, D9~, D10~, D11~, D12, D13, A0 (VCC), A1 (VCC), A2 (VCC), A3 (VCC), A4 (VCC), A5 (VCC), 5V (VCC), 3V3 (VCC), GND (GND), GND (GND), VIN (VCC) |
| ESP32 DevKit (CH340) | Controladores | Kit 3 | 3V3 (VCC), EN, D36, D39, D34, D35, D32, D33, D25, D26, D27, D14, D12, D13, GND (GND), 5V (VCC), D23, D22, TX0, RX0, D21, D19, D18, D5, D17, D16, D4, D0, D2 |
| Sensor ultrasónico HC-SR04 | Sensores | Kit 1, Kit 2, Kit 3 | VCC (VCC), GND (GND), Trig, Echo |
| Sensor DHT11 (temp/humedad) | Sensores | Kit 3 | VCC (VCC), DATA, GND (GND) |
| Módulo LDR (sensor de luz) | Sensores | Kit 3 | VCC (VCC), AO, GND (GND) |
| Sensor de presencia HC-SR501 | Sensores | Kit 3 | VCC (VCC), OUT, GND (GND) |
| Sensor de línea IR (TCRT5000) | Sensores | Kit 1, Kit 2 | VCC (VCC), OUT, GND (GND) |

| Componente | Categoría | Kit | Pines (en el orden del dibujo) |
|---|---|---|---|
| Sensor IR de obstáculos | Sensores | Kit 2 | VCC (VCC), OUT, GND (GND) |
| Módulo de aplausos LM393 | Sensores | Kit 1, Kit 3 | VCC (VCC), GND (GND), DO, AO |
| Receptor IR + control remoto | Sensores | Kit 2 | VCC (VCC), OUT, GND (GND) |
| Motor DC | Actuadores | Kit 1 | + (VCC), - (GND) |
| Motorreductor TT | Actuadores | Kit 1, Kit 2 | + (VCC), - (GND) |
| Servomotor | Actuadores | Kit 1, Kit 2, Kit 3 | GND (GND), VCC (VCC), Señal |
| Módulo relé 1 canal | Módulos | Kit 3 | IN, VCC (VCC), GND (GND), NO, COM, NC |
| Driver de motores L298N | Módulos | Kit 1 | +V (VCC), GND (GND), ENA~, IN1, IN2, IN3, IN4, ENB~, OUT1, OUT2, OUT3, OUT4 |
| Shield de motores L293D | Módulos | Kit 2 | 5V (VCC), GND (GND), D3, D5, D6~, D9~, D10~, D11~, M1, M2, M3, M4 |
| Pantalla LCD 16x2 (I2C) | Visualización | Kit 3 | VCC (VCC), GND (GND), SDA, SCL |
| Multímetro digital | Instrumentos | Kit 1, Kit 2, Kit 3 | COM (negra) (GND), V (roja) (VCC), A |

Las etiquetas K1-K2… de la paleta indican de qué kit(s) es cada componente, igual que las insignias «Kit N» de las prácticas guiadas.

## 3.4 Colocar, mover, rotar y eliminar

• Colocar: arrastra un componente desde la paleta y suéltalo en la mesa (queda centrado donde lo sueltes).

• Seleccionar: clic en el componente. Aparecen sus propiedades en el panel derecho.

• Mover: arrastra el cuerpo del componente.

• Rotar: con el componente seleccionado, pulsa « Rotar» (gira 90° cada vez).

• Eliminar: con el componente o el cable seleccionado, pulsa Supr (en los cables también vale clic derecho). « Limpiar mesa» borra todo y « Reiniciar» vuelve al estado inicial.

• Vista: la rueda del ratón hace zoom (se muestra el porcentaje) y arrastrar el fondo desplaza la vista (paneo). Los botones - / + también hacen zoom.

Mientras el laboratorio está abierto no se selecciona texto (no aparece el resaltado azul al arrastrar cables sobre los componentes).

## 3.5 Tender cables entre pines

• Haz clic en un pin (círculo del borde del componente) y luego clic en otro pin: se tiende el cable. Mientras arrastras se ve el cable provisional.

• Los pines de alimentación se distinguen por color: rojo = +/VCC/5V, gris = GND, amarillo = señal.

• Color del cable: el selector de la paleta permite elegir (Automático, rojo, negro, amarillo, verde, azul, blanco). Con «Automático»: + -> rojo, GND -> negro, señal -> amarillo.

• Clic en un cable lo selecciona (resaltado); Supr o clic derecho lo elimina.

## 3.6 Alimentación y medición

• Pulsa «[ON/OFF] Alimentación: ON»: es el interruptor general. Con OFF, todas las fuentes y los microcontroladores quedan en 0 V.

• Fuente regulable 0–12 V: ajusta el voltaje con el deslizador del panel de propiedades; su pantalla muestra el valor.

• Arduino UNO / ESP32: se alimentan por USB (automático al ponerlos), por VIN o por el pin 5V, y entregan tensión entre 5V (o 3V3) y GND — p. ej. 5V<->GND = 5,000 V en el multímetro (GND y GND2

son el mismo nodo interno). El ESP32 trabaja a 3,3 V.

• Multímetro digital: conecta COM (sonda negra) y V (sonda roja) y elige el modo en sus propiedades: V (voltaje, en paralelo), A (corriente, en serie con el circuito),  (resistencia entre puntas; con el circuito alimentado muestra «OL»).

## 3.7 Propiedades editables por componente

| Componente | Propiedades que puedes ajustar en el panel derecho |
|---|---|
| Fuente regulable | Voltaje 0–12 V (deslizador), encendido/apagado |
| Resistencia | Valor en ohmios (R) |
| Potenciómetro | Posición (se gira arrastrando la perilla directamente en la mesa) |
| Multímetro | Modo: V / A / |
| HC-SR04 | Distancia detectada (cm) — el programa la leerá con pulseIn() |
| LDR | Nivel de luz 0–1 (afecta la salida analógica AO) |
| DHT11 | Temperatura (°C) y humedad (%) que entregará el sensor |
| HC-SR501 / línea IR / IR obstáculos | Estado (movimiento, línea negra, obstáculo) |
| Arduino / ESP32 | Alimentación por USB (propiedad usb) |

## 3.8 Programación: editor, intérprete y monitor serial

• Escribe el código en el editor (con resaltado de sintaxis) o cárgalo desde una práctica guiada (« Cargar código en el editor»).

• Pulsa «> Cargar y ejecutar»: el intérprete transpila el código Arduino/ESP32 (C++ -> JS) y lo ejecuta contra los nodos eléctricos reales del circuito. «[] Detener» detiene el programa.

• Los mensajes de Serial.print/println aparecen en el monitor serial del panel derecho.

• El programa solo funciona si el cableado es correcto: por ejemplo, analogRead(A0) lee el voltaje real del nodo A0; digitalWrite pone 5 V (o 0 V) en el pin con su resistencia interna; un LED conectado a un pin de salida se enciende de verdad.

### API soportada por el intérprete

| Grupo | Funciones / clases |
|---|---|
| Pines digitales | pinMode, digitalWrite, digitalRead (constantes HIGH, LOW, INPUT, OUTPUT, INPUT_PULLUP) |
| Pines analógicos | analogRead (A0–A5 = 14–19), analogWrite (PWM en pines ~), map, constrain |
| Tiempo | delay, delayMicroseconds, millis |
| Tono | tone(pin, Hz[, duración]), noTone(pin) |
| Pulso / distancia | pulseIn(pin, HIGH) — usado con el HC-SR04 |
| Serial | Serial.begin, Serial.print, Serial.println |
| Servo | Servo: attach(pin), write(ángulo 0–180), writeMicroseconds |
| LCD 16x2 (I2C) | LiquidCrystal_I2C lcd(0x27, 16, 2): begin, setCursor, print, clear |
| LCD 16x2 (paralelo, heredada) | LiquidCrystal lcd(rs, e, d4, d5, d6, d7) |
| Sensores | DHT: readTemperature/readHumidity - IRrecv: enableIRIn/decode |

### Ejemplo mínimo (LED en D9)

```
int led = 9;

void setup() {
  pinMode(led, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(led, HIGH);   // 5 V en D9 -> el LED se enciende
  Serial.println("LED ON");
  delay(1000);
  digitalWrite(led, LOW);
  Serial.println("LED OFF");
  delay(1000);
}
```

## 3.9 La pantalla LCD 16x2 con módulo I2C

• En el kit 3 y en el lab la LCD viene con el módulo I2C (PCF8574, dirección habitual 0x27) ya soldado: solo tiene 4 pines: VCC, GND, SDA, SCL.

• Conexiones: VCC -> 5 V, GND -> GND, SDA -> A4, SCL -> A5 en el Arduino UNO; en el ESP32 SDA -> D21 (GPIO21), SCL -> D22 (GPIO22).

• Código: LiquidCrystal_I2C lcd(0x27, 16, 2); lcd.begin(16, 2); lcd.setCursor(0, 0); lcd.print("HOLA MUNDO");

• La simulación solo dibuja el texto en la pantalla virtual si la LCD está alimentada (VCC/GND) y SDA/SCL llegan a los pines I2C del microcontrolador; si no, aparece un aviso explicándolo.

## 3.10 Avisos inteligentes (validación en español)

El panel « Avisos del circuito» detecta y explica en tiempo real:

• Cortocircuito (cable directo entre + y - de una fuente)

• LED conectado con polaridad invertida (no enciende)

• LED sin resistencia limitadora o con más de 30 mA (riesgo de daño)

• Resistencia que disipa más de 250 mW

• Componentes sin alimentación (p. ej. LCD sin VCC/GND)

• LCD encendida pero sin SDA/SCL hacia el microcontrolador

• Pines del Arduino/ESP32 sin conectar y pines de salida con más de 40 mA

# 4. Prácticas guiadas (11 proyectos)

Desde « Prácticas guiadas» se abre el índice: cada tarjeta muestra el título, el objetivo y las insignias «Kit N» de los kits a los que pertenecen sus componentes. Al entrar en una práctica ves el objetivo, la lista de componentes con su kit, el diagrama de conexiones, los pasos, la explicación y el código de ejemplo, con botones para colocar los componentes en la mesa (sin conectarlos: tú debes tender los cables) y para cargar el código en el editor.

## 4.1 Práctica 1: Encendido de un LED con resistencia

Objetivo: Aprender a limitar la corriente de un LED con una resistencia usando la Ley de Ohm (V = I × R) y comprobar los valores con el multímetro.

Kits de los componentes: Kit 1, Kit 2, Kit 3

| Componente | Kit | Propiedades iniciales |
|---|---|---|
| Fuente regulable 0–12 V | Kit 1, Kit 2, Kit 3 | volt=5, on=True |
| Resistencia | Kit 1, Kit 3 | R=220 |
| LED | Kit 1 | — |
| Multímetro digital | Kit 1, Kit 2, Kit 3 | mode=V |

### Diagrama de conexiones (cables que debes tender)

| Desde | Hasta |
|---|---|
| Fuente regulable 0–12 V - pin plus | Resistencia - pin a |
| Resistencia - pin b | LED - pin anodo |
| LED - pin catodo | Fuente regulable 0–12 V - pin minus |

### Pasos

1. Coloca la fuente regulable en 5 V y enciende la alimentación general ([ON/OFF]).
2. Conecta el terminal + de la fuente a un extremo de la resistencia (cable rojo).
3. Conecta el otro extremo de la resistencia al ÁNODO (+) del LED.
4. Conecta el CÁTODO (-) del LED al terminal - de la fuente (cable negro).
5. Observa el LED: debe encenderse. Mídelo: corriente (5 V - 2 V) / 220 13,6 mA.
6. Prueba con el multímetro en modo A (en serie) y en modo V (en paralelo sobre el LED y la resistencia).
7. Experimento: invierte el LED. Verás el aviso de polaridad invertida y el LED no enciende.
8. Experimento: quita la resistencia. El sistema advierte que el LED puede dañarse por exceso de corriente.

### Funcionamiento

Un LED necesita unos 2 V para conducir, pero si se conecta directo a 5 V la corriente sería enorme y se quema. La resistencia de 220 limita la corriente a unos 13 mA: I = (5 - 2) / 220 0,0136 A. Sin LED, una resistencia de 220 a 5 V consumiría 5/220 22,7 mA (Ley de Ohm).

### Código de ejemplo

```
// Proyecto 1: LED + resistencia (no requiere código, es un circuito analógico)
// El LED se enciende solo al alimentar el circuito.
// Prueba a medir con el multímetro:
//   - Voltaje en paralelo con la resistencia y con el LED
//   - Corriente en serie (modo A del multímetro)
void setup() {
  Serial.begin(9600);
  Serial.println("Circuito LED + 220 ohm a 5V");
  Serial.println("I esperado = (5 - 2) / 220 = 13.6 mA");
}

void loop() {
}
```

## 4.2 Práctica 2: Control de brillo de un LED con potenciómetro (PWM)

Objetivo: Usar una entrada analógica (A0) para leer un potenciómetro y controlar el brillo de un LED con PWM (analogWrite).

Kits de los componentes: Kit 1, Kit 2, Kit 3

| Componente | Kit | Propiedades iniciales |
|---|---|---|
| Arduino UNO | Kit 1, Kit 2 | — |
| Potenciómetro 10 k | Kit 1 | — |
| Resistencia | Kit 1, Kit 3 | R=220 |
| LED | Kit 1 | — |

### Diagrama de conexiones (cables que debes tender)

| Desde | Hasta |
|---|---|
| Arduino UNO - pin 5V | Potenciómetro 10 k - pin a |
| Arduino UNO - pin GND | Potenciómetro 10 k - pin b |
| Potenciómetro 10 k - pin w | Arduino UNO - pin A0 |
| Arduino UNO - pin D9 | Resistencia - pin a |
| Resistencia - pin b | LED - pin anodo |
| LED - pin catodo | Arduino UNO - pin GND |

### Pasos

1. Coloca el Arduino UNO (alimentado por USB), el potenciómetro, la resistencia de 220 y el LED.
2. Alimenta el potenciómetro: A -> 5V, B -> GND (pines del Arduino).
3. Conecta el cursor (W) del potenciómetro a la entrada analógica A0.
4. Conecta el pin D9~ (PWM) a la resistencia de 220 .
5. Conecta la resistencia al ÁNODO del LED y el CÁTODO a GND.
6. Carga el código de ejemplo con «> Cargar y ejecutar».
7. Gira el potenciómetro: el brillo del LED debe cambiar suavemente de apagado a máximo.

### Funcionamiento

El potenciómetro funciona como divisor de tensión: el cursor entrega entre 0 y 5 V según su posición. analogRead(A0) lo convierte a un valor de 0 a 1023. Con map() se traduce a 0–255 y analogWrite(D9) genera una señal PWM cuyo promedio de voltaje controla el brillo del LED. El motor de simulación calcula la corriente real del LED en cada posición.

### Código de ejemplo

```
// Proyecto 2: brillo del LED controlado por potenciómetro
int potenciometro = A0;   // cursor del potenciómetro
int led = 9;              // pin PWM ~9

void setup() {
  pinMode(led, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int valor = analogRead(potenciometro);      // 0..1023
  int brillo = map(valor, 0, 1023, 0, 255);   // 0..255
  analogWrite(led, brillo);                    // PWM
  Serial.print("Potenciometro: ");
  Serial.println(valor);
  delay(50);
}
```

## 4.3 Práctica 3: Pantalla LCD 16x2 (I2C) mostrando «HOLA MUNDO»

Objetivo: Conectar la pantalla LCD 16x2 con su módulo I2C usando solo 4 cables (VCC, GND, SDA, SCL) y mostrar un texto con la librería LiquidCrystal_I2C.

Kits de los componentes: Kit 1, Kit 2, Kit 3

| Componente | Kit | Propiedades iniciales |
|---|---|---|
| Arduino UNO | Kit 1, Kit 2 | — |
| Pantalla LCD 16x2 (I2C) | Kit 3 | — |

### Diagrama de conexiones (cables que debes tender)

| Desde | Hasta |
|---|---|
| Arduino UNO - pin 5V | Pantalla LCD 16x2 (I2C) - pin vcc |
| Arduino UNO - pin GND | Pantalla LCD 16x2 (I2C) - pin gnd |
| Arduino UNO - pin A4 | Pantalla LCD 16x2 (I2C) - pin sda |
| Arduino UNO - pin A5 | Pantalla LCD 16x2 (I2C) - pin scl |

### Pasos

1. Coloca el Arduino UNO y la pantalla LCD 16x2 con módulo I2C (los 4 pines de arriba: VCC, GND, SDA, SCL).
2. Alimenta la pantalla: VCC -> 5V y GND -> GND del Arduino (cables rojo y negro).
3. Conecta el bus I2C: SDA -> A4 y SCL -> A5 (en el UNO, A4/A5 son los pines I2C).
4. Carga el código de ejemplo con «> Cargar y ejecutar».
5. La pantalla debe mostrar «HOLA MUNDO».
6. Experimento: desconecta VCC. El sistema avisa que la LCD no recibe alimentación.
7. Experimento: desconecta SDA. El aviso indica que faltan SDA/SCL hacia el microcontrolador.

### Funcionamiento

La LCD 16x2 del kit trae soldado el módulo I2C (PCF8574, dirección habitual 0x27): traduce el bus serie I2C a las señales del controlador HD44780, así que solo ocupa 2 pines de datos (SDA/SCL) además de la alimentación. La simulación comprueba que VCC/GND estén alimentados y que SDA/SCL lleguen a los pines I2C del Arduino (A4/A5); solo entonces lcd.print() dibuja el texto en la pantalla virtual.

### Código de ejemplo

```
// Proyecto 3: HOLA MUNDO en LCD 16x2 con módulo I2C
#include <Wire.h>
#include <LiquidCrystal_I2C.h>   // (simulada)

LiquidCrystal_I2C lcd(0x27, 16, 2);  // dirección I2C, 16 columnas, 2 filas

void setup() {
  lcd.begin(16, 2);
  lcd.setCursor(0, 0);
  lcd.print("HOLA MUNDO");
}

void loop() {
}
```

## 4.4 Práctica 4: LCD (I2C) mostrando el valor del potenciómetro en vivo

Objetivo: Leer un potenciómetro con analogRead y mostrar su valor (0–1023) en tiempo real en la LCD con módulo I2C.

Kits de los componentes: Kit 1, Kit 2, Kit 3

| Componente | Kit | Propiedades iniciales |
|---|---|---|
| Arduino UNO | Kit 1, Kit 2 | — |
| Pantalla LCD 16x2 (I2C) | Kit 3 | — |
| Potenciómetro 10 k | Kit 1 | — |

### Diagrama de conexiones (cables que debes tender)

| Desde | Hasta |
|---|---|
| Arduino UNO - pin 5V | Pantalla LCD 16x2 (I2C) - pin vcc |
| Arduino UNO - pin GND | Pantalla LCD 16x2 (I2C) - pin gnd |
| Arduino UNO - pin A4 | Pantalla LCD 16x2 (I2C) - pin sda |
| Arduino UNO - pin A5 | Pantalla LCD 16x2 (I2C) - pin scl |
| Arduino UNO - pin 5V | Potenciómetro 10 k - pin a |
| Arduino UNO - pin GND | Potenciómetro 10 k - pin b |
| Potenciómetro 10 k - pin w | Arduino UNO - pin A0 |

### Pasos

1. Monta la LCD con módulo I2C: VCC -> 5V, GND -> GND, SDA -> A4, SCL -> A5.
2. Añade el potenciómetro: A -> 5V, B -> GND, cursor W -> A0.
3. Carga y ejecuta el código.
4. Gira el potenciómetro: la LCD debe mostrar «VALOR: xxx» cambiando en vivo entre 0 y 1023.

### Funcionamiento

El potenciómetro como divisor de tensión entrega en W un voltaje proporcional a su posición. analogRead(A0) lo mide (0–1023). El programa imprime ese valor en la segunda fila de la LCD; al girar el mando, el valor cambia dinámicamente en la pantalla simulada.

### Código de ejemplo

```
// Proyecto 4: valor del potenciómetro en la LCD (I2C)
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
int potenciometro = A0;

void setup() {
  lcd.begin(16, 2);
  lcd.print("POTENCIOMETRO");
  Serial.begin(9600);
}

void loop() {
  int valor = analogRead(potenciometro);
  lcd.setCursor(0, 1);
  lcd.print("VALOR: ");
  lcd.print(valor);
  lcd.print("    ");           // limpia restos
  Serial.println(valor);
  delay(100);
}
```

## 4.5 Práctica 5: Tono del buzzer controlado por potenciómetro

Objetivo: Generar sonido con tone() y variar su frecuencia según la posición de un potenciómetro leído en A0.

Kits de los componentes: Kit 1, Kit 2, Kit 3

| Componente | Kit | Propiedades iniciales |
|---|---|---|
| Arduino UNO | Kit 1, Kit 2 | — |
| Buzzer | Kit 3 | — |
| Potenciómetro 10 k | Kit 1 | — |

### Diagrama de conexiones (cables que debes tender)

| Desde | Hasta |
|---|---|
| Arduino UNO - pin D8 | Buzzer - pin plus |
| Buzzer - pin minus | Arduino UNO - pin GND |
| Arduino UNO - pin 5V | Potenciómetro 10 k - pin a |
| Arduino UNO - pin GND | Potenciómetro 10 k - pin b |
| Potenciómetro 10 k - pin w | Arduino UNO - pin A0 |

### Pasos

1. Coloca Arduino, buzzer y potenciómetro.
2. Conecta el buzzer: + -> pin D8, - -> GND.
3. Conecta el potenciómetro: A -> 5V, B -> GND, W -> A0.
4. Carga y ejecuta el código.
5. Gira el potenciómetro: escucharás cómo cambia el tono (200 Hz a 2000 Hz) y el simulador muestra la frecuencia generada.

### Funcionamiento

tone(pin, frecuencia) genera una onda cuadrada en el pin. El potenciómetro fija la frecuencia entre 200 y 2000 Hz mediante analogRead + map. El buzzer simulado reproduce el sonido real con WebAudio y muestra visualmente cuándo está vibrando.

### Código de ejemplo

```
// Proyecto 5: frecuencia del buzzer según el potenciómetro
int potenciometro = A0;
int buzzer = 8;

void setup() {
  pinMode(buzzer, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int valor = analogRead(potenciometro);          // 0..1023
  int frecuencia = map(valor, 0, 1023, 200, 2000); // 200..2000 Hz
  tone(buzzer, frecuencia);
  Serial.print("Frecuencia: ");
  Serial.print(frecuencia);
  Serial.println(" Hz");
  delay(80);
}
```

## 4.6 Práctica 6: Sensor ultrasónico HC-SR04 que mueve un servomotor

Objetivo: Medir distancia con pulseIn() y orientar un servomotor según la distancia detectada (como el 'radar' de un robot).

Kits de los componentes: Kit 1, Kit 2, Kit 3

| Componente | Kit | Propiedades iniciales |
|---|---|---|
| Arduino UNO | Kit 1, Kit 2 | — |
| Sensor ultrasónico HC-SR04 | Kit 1, Kit 2, Kit 3 | — |
| Servomotor | Kit 1, Kit 2, Kit 3 | — |

### Diagrama de conexiones (cables que debes tender)

| Desde | Hasta |
|---|---|
| Arduino UNO - pin 5V | Sensor ultrasónico HC-SR04 - pin vcc |
| Arduino UNO - pin GND | Sensor ultrasónico HC-SR04 - pin gnd |
| Arduino UNO - pin D7 | Sensor ultrasónico HC-SR04 - pin trig |
| Arduino UNO - pin D6 | Sensor ultrasónico HC-SR04 - pin echo |
| Arduino UNO - pin 5V | Servomotor - pin vcc |
| Arduino UNO - pin GND | Servomotor - pin gnd |
| Arduino UNO - pin D9 | Servomotor - pin sig |

### Pasos

1. Coloca Arduino, HC-SR04 y el servomotor.
2. Alimenta el sensor: VCC -> 5V, GND -> GND.
3. Conecta Trig -> D7 y Echo -> D6.
4. Alimenta el servo (VCC -> 5V, GND -> GND) y su señal -> D9~.
5. Carga y ejecuta el código.
6. En el panel de propiedades del HC-SR04, mueve el deslizador «distancia»: el servo debe girar proporcionalmente (cerca -> 0°, lejos -> 180°).

### Funcionamiento

El HC-SR04 emite un pulso ultrasónico por Trig; Echo devuelve un pulso cuyo ancho es proporcional al tiempo de ida y vuelta: distancia(cm) = tiempo(µs)/58. pulseIn() mide ese pulso y el programa convierte la distancia en un ángulo con map() para el servo (Servo.write). En la simulación puedes variar la distancia detectada con el control del sensor.

### Código de ejemplo

```
// Proyecto 6: radar con HC-SR04 + servo
#include <Servo.h>

Servo servo;
int trig = 7;
int echo = 6;

void setup() {
  servo.attach(9);
  pinMode(trig, OUTPUT);
  pinMode(echo, INPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);

  long duracion = pulseIn(echo, HIGH);   // µs
  int distancia = duracion / 58;          // cm

  int angulo = map(distancia, 2, 200, 0, 180);
  angulo = constrain(angulo, 0, 180);
  servo.write(angulo);

  Serial.print("Distancia: ");
  Serial.print(distancia);
  Serial.println(" cm");
  delay(150);
}
```

## 4.7 Práctica 7: Estación climática: DHT11 + LCD I2C en ESP32

Objetivo: Leer temperatura y humedad con el DHT11 y mostrarlas en la pantalla LCD 16x2 con módulo I2C, todo con un ESP32.

Kits de los componentes: Kit 3

| Componente | Kit | Propiedades iniciales |
|---|---|---|
| ESP32 DevKit (CH340) | Kit 3 | — |
| Sensor DHT11 (temp/humedad) | Kit 3 | temp=27, humedad=55 |
| Pantalla LCD 16x2 (I2C) | Kit 3 | — |

### Diagrama de conexiones (cables que debes tender)

| Desde | Hasta |
|---|---|
| ESP32 DevKit (CH340) - pin 3V3 | Sensor DHT11 (temp/humedad) - pin vcc |
| ESP32 DevKit (CH340) - pin GND | Sensor DHT11 (temp/humedad) - pin gnd |
| ESP32 DevKit (CH340) - pin D4 | Sensor DHT11 (temp/humedad) - pin data |
| ESP32 DevKit (CH340) - pin 5V | Pantalla LCD 16x2 (I2C) - pin vcc |
| ESP32 DevKit (CH340) - pin GND | Pantalla LCD 16x2 (I2C) - pin gnd |
| ESP32 DevKit (CH340) - pin D21 | Pantalla LCD 16x2 (I2C) - pin sda |
| ESP32 DevKit (CH340) - pin D22 | Pantalla LCD 16x2 (I2C) - pin scl |

### Pasos

1. Coloca el ESP32, el sensor DHT11 y la LCD 16x2 con módulo I2C.
2. Alimenta el DHT11: VCC -> 3V3, GND -> GND y DATA -> D4.
3. Conecta la LCD: VCC -> 5V, GND -> GND, SDA -> D21 y SCL -> D22 (pines I2C del ESP32).
4. Carga y ejecuta el código.
5. La LCD muestra «T: xxC» y «H: xx%». Cambia la temperatura/humedad en el panel de propiedades del DHT11 y obsévalo en vivo.
6. Experimento: desconecta SDA de la LCD y verás el aviso de bus I2C no conectado.

### Funcionamiento

El DHT11 mide temperatura y humedad y entrega el dato por un pin digital (aquí D4). La LCD con módulo I2C se conecta solo con VCC, GND, SDA y SCL: en el ESP32 los pines I2C por defecto son GPIO21 (SDA) y GPIO22 (SCL), que en el simulador son D21 y D22. El programa lee el sensor con la librería DHT y escribe las dos filas de la pantalla con LiquidCrystal_I2C.

### Código de ejemplo

```
// Proyecto 7: estación climática ESP32 + DHT11 + LCD I2C
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

DHT dht(4, DHT11);
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  Serial.begin(115200);
  dht.begin();
  lcd.begin(16, 2);
  lcd.print("ESTACION");
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  lcd.setCursor(0, 0);
  lcd.print("T: "); lcd.print(t); lcd.print("C   ");
  lcd.setCursor(0, 1);
  lcd.print("H: "); lcd.print(h); lcd.print("%   ");
  Serial.print(t); Serial.print(" C / "); Serial.print(h); Serial.println(" %");
  delay(500);
}
```

## 4.8 Práctica 8: Relé que enciende una lámpara (LED de potencia)

Objetivo: Controlar un circuito de potencia (fuente + LED) con un relé activado desde un pin digital del Arduino, como una lámpara real.

Kits de los componentes: Kit 1, Kit 2, Kit 3

| Componente | Kit | Propiedades iniciales |
|---|---|---|
| Arduino UNO | Kit 1, Kit 2 | — |
| Módulo relé 1 canal | Kit 3 | — |
| Fuente regulable 0–12 V | Kit 1, Kit 2, Kit 3 | volt=9, on=True |
| Resistencia | Kit 1, Kit 3 | R=470 |
| LED | Kit 1 | — |

### Diagrama de conexiones (cables que debes tender)

| Desde | Hasta |
|---|---|
| Arduino UNO - pin 5V | Módulo relé 1 canal - pin vcc |
| Arduino UNO - pin GND | Módulo relé 1 canal - pin gnd |
| Arduino UNO - pin D7 | Módulo relé 1 canal - pin in |
| Fuente regulable 0–12 V - pin plus | Módulo relé 1 canal - pin com |
| Módulo relé 1 canal - pin no | Resistencia - pin a |
| Resistencia - pin b | LED - pin anodo |
| LED - pin catodo | Fuente regulable 0–12 V - pin minus |

### Pasos

1. Coloca Arduino, módulo relé, fuente de 9 V, resistencia de 470 y LED.
2. Alimenta el relé: VCC -> 5V, GND -> GND, y su entrada IN -> D7.
3. Conecta la fuente de 9 V al relé: + -> COM.
4. Del contacto NO del relé salen la resistencia (470 ) -> ánodo del LED -> cátodo al - de la fuente.
5. Carga y ejecuta el código: el relé debe conmutar cada 2 s y el LED encenderse/apagarse.
6. Experimento: deja IN sin conectar o a LOW; el LED debe quedar apagado (COM->NC abierto para la carga).

### Funcionamiento

Un relé es un interruptor electromecánico: la bobina (IN) cierra el contacto COM->NO cuando recibe nivel alto. Así un pin del Arduino (pocos mA) controla un circuito de potencia (9 V, LED con su resistencia). En la simulación el LED del módulo relé indica el estado y el LED de carga se comporta eléctricamente como en la realidad: I = (9 - 2) / 470 15 mA.

### Código de ejemplo

```
// Proyecto 8: lámpara con relé
int rele = 7;

void setup() {
  pinMode(rele, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(rele, HIGH);   // bobina activada: COM->NO cerrado, lámpara ON
  Serial.println("LAMPA ON");
  delay(2000);
  digitalWrite(rele, LOW);    // relé liberado: lámpara OFF
  Serial.println("LAMPA OFF");
  delay(2000);
}
```

## 4.9 Práctica 9: Servomotor controlado con un potenciómetro

Objetivo: Mover un servomotor a cualquier ángulo (0–180°) girando un potenciómetro, combinando analogRead, map y Servo.

Kits de los componentes: Kit 1, Kit 2, Kit 3

| Componente | Kit | Propiedades iniciales |
|---|---|---|
| Arduino UNO | Kit 1, Kit 2 | — |
| Potenciómetro 10 k | Kit 1 | — |
| Servomotor | Kit 1, Kit 2, Kit 3 | — |

### Diagrama de conexiones (cables que debes tender)

| Desde | Hasta |
|---|---|
| Arduino UNO - pin 5V | Potenciómetro 10 k - pin a |
| Arduino UNO - pin GND | Potenciómetro 10 k - pin b |
| Potenciómetro 10 k - pin w | Arduino UNO - pin A0 |
| Arduino UNO - pin 5V | Servomotor - pin vcc |
| Arduino UNO - pin GND | Servomotor - pin gnd |
| Arduino UNO - pin D9 | Servomotor - pin sig |

### Pasos

1. Coloca el Arduino, el potenciómetro y el servomotor.
2. Alimenta el potenciómetro: A -> 5V, B -> GND, cursor W -> A0.
3. Alimenta el servo: VCC -> 5V, GND -> GND y su señal (cable naranja) -> D9~.
4. Carga y ejecuta el código.
5. Gira el potenciómetro de tope a tope: el brazo del servo debe barrer de 0° a 180°.

### Funcionamiento

El servo lee pulsos PWM en su pin de señal: analogWrite/attach generan esos pulsos y Servo.write(ángulo) fija la posición. El potenciómetro entrega 0–5 V en W; analogRead(A0) lo convierte a 0–1023 y map() lo traduce a 0–180°. En la simulación el brazo del servo gira visualmente y la corriente del servo se calcula según el ángulo.

### Código de ejemplo

```
// Proyecto 9: servo + potenciómetro
#include <Servo.h>

Servo servo;
int potenciometro = A0;
int pinServo = 9;

void setup() {
  servo.attach(pinServo);
  Serial.begin(9600);
}

void loop() {
  int valor = analogRead(potenciometro);          // 0..1023
  int angulo = map(valor, 0, 1023, 0, 180);       // 0..180 grados
  servo.write(angulo);
  Serial.print("Angulo: ");
  Serial.println(angulo);
  delay(60);
}
```

## 4.10 Práctica 10: Alarma de proximidad: HC-SR04 + buzzer

Objetivo: Hacer sonar un buzzer cuando un objeto se acerque a menos de 15 cm, midiendo distancia con el ultrasónico HC-SR04.

Kits de los componentes: Kit 1, Kit 2, Kit 3

| Componente | Kit | Propiedades iniciales |
|---|---|---|
| Arduino UNO | Kit 1, Kit 2 | — |
| Sensor ultrasónico HC-SR04 | Kit 1, Kit 2, Kit 3 | distancia=40 |
| Buzzer | Kit 3 | — |

### Diagrama de conexiones (cables que debes tender)

| Desde | Hasta |
|---|---|
| Arduino UNO - pin 5V | Sensor ultrasónico HC-SR04 - pin vcc |
| Arduino UNO - pin GND | Sensor ultrasónico HC-SR04 - pin gnd |
| Arduino UNO - pin D7 | Sensor ultrasónico HC-SR04 - pin trig |
| Arduino UNO - pin D6 | Sensor ultrasónico HC-SR04 - pin echo |
| Arduino UNO - pin D8 | Buzzer - pin plus |
| Buzzer - pin minus | Arduino UNO - pin GND |

### Pasos

1. Coloca Arduino, HC-SR04 y buzzer.
2. Alimenta el sensor: VCC -> 5V, GND -> GND; Trig -> D7 y Echo -> D6.
3. Conecta el buzzer: + -> D8 y - -> GND.
4. Carga y ejecuta el código.
5. En el panel de propiedades del HC-SR04 baja la distancia a menos de 15 cm: el buzzer debe sonar. Súbela: se silencia.

### Funcionamiento

El HC-SR04 mide distancia por ultrasonido: Trig emite un pulso de 10 µs y Echo devuelve un pulso cuyo ancho es proporcional al tiempo de ida y vuelta (cm = µs/58). pulseIn() mide ese ancho. Si la distancia es menor que el umbral, tone(D8, 1200) hace sonar el buzzer; si no, noTone() lo silencia. En la simulación varías la distancia con el control del sensor.

### Código de ejemplo

```
// Proyecto 10: alarma de proximidad HC-SR04 + buzzer
int trig = 7;
int echo = 6;
int buzzer = 8;

void setup() {
  pinMode(trig, OUTPUT);
  pinMode(echo, INPUT);
  pinMode(buzzer, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);

  long us = pulseIn(echo, HIGH);   // duración del pulso de eco (µs)
  int cm = us / 58;                // distancia en cm

  if (cm > 0 && cm < 15) {
    tone(buzzer, 1200);            // ¡objeto cerca!
    Serial.print("ALARMA a ");
    Serial.print(cm);
    Serial.println(" cm");
  } else {
    noTone(buzzer);                // zona despejada
  }
  delay(100);
}
```

## 4.11 Práctica 11: Luz automática nocturna con LDR

Objetivo: Encender un LED automáticamente cuando oscurezca, leyendo el módulo LDR (fotoresistencia) con analogRead.

Kits de los componentes: Kit 1, Kit 2, Kit 3

| Componente | Kit | Propiedades iniciales |
|---|---|---|
| Arduino UNO | Kit 1, Kit 2 | — |
| Módulo LDR (sensor de luz) | Kit 3 | luz=0.8 |
| Resistencia | Kit 1, Kit 3 | R=220 |
| LED | Kit 1 | — |

### Diagrama de conexiones (cables que debes tender)

| Desde | Hasta |
|---|---|
| Arduino UNO - pin 5V | Módulo LDR (sensor de luz) - pin vcc |
| Arduino UNO - pin GND | Módulo LDR (sensor de luz) - pin gnd |
| Módulo LDR (sensor de luz) - pin ao | Arduino UNO - pin A0 |
| Arduino UNO - pin D8 | Resistencia - pin a |
| Resistencia - pin b | LED - pin anodo |
| LED - pin catodo | Arduino UNO - pin GND |

### Pasos

1. Coloca Arduino, módulo LDR, resistencia de 220 y LED.
2. Alimenta el LDR: VCC -> 5V, GND -> GND y su salida analógica AO -> A0.
3. Conecta D8 -> resistencia -> ánodo del LED y cátodo -> GND.
4. Carga y ejecuta el código.
5. En el panel de propiedades del LDR mueve el deslizador «luz»: con poca luz (valor bajo en A0) el LED se enciende; con mucha luz se apaga.

### Funcionamiento

El módulo LDR lleva una fotoresistencia en un divisor de tensión: su salida AO entrega un voltaje proporcional a la luz recibida (más luz -> más voltaje). analogRead(A0) lo mide (0–1023) y el programa compara contra un umbral: si hay poca luz (valor < 400), digitalWrite(D8, HIGH) enciende el LED, como una lámpara nocturna automática. La resistencia de 220 protege el LED.

### Código de ejemplo

```
// Proyecto 11: luz automática con LDR
int ldr = A0;        // salida AO del módulo LDR
int led = 8;

void setup() {
  pinMode(led, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int luz = analogRead(ldr);        // 0..1023 (más luz = valor más alto)
  if (luz < 400) {
    digitalWrite(led, HIGH);        // oscuro -> enciende la luz
    Serial.println("NOCHE: LED ON");
  } else {
    digitalWrite(led, LOW);         // hay luz -> apaga
    Serial.println("DIA: LED OFF");
  }
  Serial.print("Luz: ");
  Serial.println(luz);
  delay(200);
}
```

# 5. Referencia de pines de las placas y la LCD

## 5.1 Arduino UNO

| Pin | Nombre | Tipo |
|---|---|---|
| D0 | D0 | E/S (PWM en los marcados ~) |
| D1 | D1 | E/S (PWM en los marcados ~) |
| D2 | D2 | E/S (PWM en los marcados ~) |
| D3 | D3~ | E/S (PWM en los marcados ~) |
| D4 | D4 | E/S (PWM en los marcados ~) |
| D5 | D5~ | E/S (PWM en los marcados ~) |
| D6 | D6~ | E/S (PWM en los marcados ~) |
| D7 | D7 | E/S (PWM en los marcados ~) |
| D8 | D8 | E/S (PWM en los marcados ~) |
| D9 | D9~ | E/S (PWM en los marcados ~) |
| D10 | D10~ | E/S (PWM en los marcados ~) |
| D11 | D11~ | E/S (PWM en los marcados ~) |
| D12 | D12 | E/S (PWM en los marcados ~) |
| D13 | D13 | E/S (PWM en los marcados ~) |
| A0 | A0 | VCC/5V |
| A1 | A1 | VCC/5V |
| A2 | A2 | VCC/5V |
| A3 | A3 | VCC/5V |
| A4 | A4 | VCC/5V |
| A5 | A5 | VCC/5V |
| 5V | 5V | VCC/5V |
| 3V3 | 3V3 | VCC/5V |
| GND | GND | GND |
| GND2 | GND | GND |
| VIN | VIN | VCC/5V |

Alimentación: USB (automático), VIN (7–12 V) o 5 V. Entrega 5 V entre 5V y GND (y 3,3 V en 3V3). Pines I2C: A4 = SDA, A5 = SCL.

## 5.2 ESP32 DevKit (CH340)

| Pin | Tipo |
|---|---|
| 3V3 | VCC (5V/3V3) |
| EN | GPIO (E/S) |
| D36 | GPIO (E/S) |
| D39 | GPIO (E/S) |
| D34 | GPIO (E/S) |
| D35 | GPIO (E/S) |
| D32 | GPIO (E/S) |
| D33 | GPIO (E/S) |

| Pin | Tipo |
|---|---|
| D25 | GPIO (E/S) |
| D26 | GPIO (E/S) |
| D27 | GPIO (E/S) |
| D14 | GPIO (E/S) |
| D12 | GPIO (E/S) |
| D13 | GPIO (E/S) |
| GND | GND |
| 5V | VCC (5V/3V3) |
| D23 | GPIO (E/S) |
| D22 | GPIO (E/S) |
| TX0 | GPIO (E/S) |
| RX0 | GPIO (E/S) |
| D21 | GPIO (E/S) |
| D19 | GPIO (E/S) |
| D18 | GPIO (E/S) |
| D5 | GPIO (E/S) |
| D17 | GPIO (E/S) |
| D16 | GPIO (E/S) |
| D4 | GPIO (E/S) |
| D0 | GPIO (E/S) |
| D2 | GPIO (E/S) |

Trabaja a 3,3 V. Pines I2C por defecto: D21 = SDA (GPIO21), D22 = SCL (GPIO22).

## 5.3 LCD 16x2 con módulo I2C

| Pin | Conexión típica |
|---|---|
| VCC | 5 V (del Arduino/ESP32 o de la fuente) |
| GND | GND |
| SDA | A4 (UNO) / D21-GPIO21 (ESP32) |
| SCL | A5 (UNO) / D22-GPIO22 (ESP32) |

## 5.4 Multímetro digital

| Pin | Uso |
|---|---|
| COM (negra) | Referencia común de las mediciones |
| V (roja) | Voltaje (en paralelo) u ohmios |
| A | Amperímetro (en serie con el circuito) |

# 6. Cómo funciona la simulación eléctrica

El motor (js/lab/engine.js) resuelve el circuito con análisis nodal modificado (MNA): cada pin es un nodo, los cables unen nodos (union-find), y cada componente aporta sus ecuaciones: resistencias, fuentes de voltaje con resistencia interna, diodos (LED con Vf 2 V y 8 ), motores, buzzers, relés, drivers L298N/L293D, sensores con salidas analógicas/digitales y los pines de salida del microcontrolador (25 internos). Los pines GND de un mismo componente son el mismo nodo interno.

• Valores de ejemplo: LED + 220 a 5 V -> 13 mA y 2 V en el LED; resistencia sola de 220 a 5 V -> 22,7 mA; potenciómetro como divisor; 5 V<->GND del Arduino alimentado = 5,000 V.

• El multímetro lee los nodos reales: V entre sus puntas, A como caída en su derivación de 0,05 , inyectando 1 V y midiendo V/I con el circuito sin alimentación.

• El intérprete lee/escribe esos mismos nodos: analogRead(A0) = 1023 × V(A0)/Vcc; pulseIn del HC-SR04 = distancia × 58 µs; digitalRead ve 0/5 V reales.

Pruebas del motor (46 aserciones, Node.js):

```
node tests/engine.test.js     # resultado esperado: 46 pasados, 0 fallados
```

# 7. Solución de problemas

| Síntoma | Causa probable | Solución |
|---|---|---|
| «No aparece el botón del laboratorio» en la web | Caché del navegador tras una actualización | Ctrl+Shift+R o abrir con ?v=N; esperar 1–2 min al build de Pages |
| El Arduino marca 0 V entre 5V y GND | Alimentación general en OFF, o sonda en un nodo no tierra | Pulsa [ON/OFF] ON; usa un pin GND (GND y GND2 son el mismo nodo) |
| El buzzer no suena | Pin sin señal (ni HIGH ni tone) o alimentación OFF | Revisa el código (tone/D8=HIGH) y [ON/OFF] ON; tras apagar/encender vuelve a sonar |
| La LCD no muestra nada | Sin VCC/GND o sin SDA/SCL al MCU | VCC->5V, GND->GND, SDA->A4/D21, SCL->A5/D22; mira el aviso del panel |
| El LED no enciende / aviso de polaridad | Ánodo y cátodo invertidos | Intercámbelos; el ánodo va hacia la resistencia/+ |
| Aviso «LED puede dañarse» | Sin resistencia en serie o corriente > 30 mA | Añade 220 en serie |
| Error de sintaxis al ejecutar | El código tiene un error de compilación | Léelo en el monitor serial (línea del error) y corrígelo en el editor |
| Los pines se ven muy juntos | Versión vieja en caché | Ctrl+Shift+R / ?v=N (en la versión actual cada placa es más grande y los pines van separados con etiquetas) |
| Al arrastrar cables se seleccionaba texto azul | — | Ya está corregido: la selección de texto está desactivada dentro del laboratorio |

# 8. Glosario

**MNA** — Modified Nodal Analysis: método para resolver circuitos eléctricos planteando ecuaciones de nodos.

**PWM** — Pulse Width Modulation: señal cuadrada cuyo ciclo de trabajo (0–255) controla potencia/brillo/posición de servo.

**I2C** — Bus serie de 2 hilos (SDA datos, SCL reloj); la LCD del kit 3 lo usa vía el módulo PCF8574 en la dirección 0x27.

**Pull-up** — Resistencia a VCC que fija un pin en alto cuando no hay nada conectado (modo INPUT_PULLUP).

**Divisor de tensión** — Dos resistencias en serie; el punto medio entrega una fracción del voltaje (lo usa el potenciómetro y el LDR).

**Relé** — Interruptor electromecánico: la bobina (IN) conmuta el contacto COM entre NC (cerrado en reposo) y NO (cerrado activado).

**HC-SR04** — Sensor ultrasónico de distancia: Trig emite un pulso y Echo devuelve uno cuyo ancho = tiempo de vuelo (cm = µs/58).

**DHT11** — Sensor de temperatura y humedad digital de un solo pin (DATA).

**LDR** — Light Dependent Resistor: fotoresistencia; su módulo da una salida analógica (AO) proporcional a la luz.

**L298N / L293D** — Drivers de motores DC: controlan velocidad (PWM en ENA/ENB o pines ~) y dirección (IN1/IN2…).

**Protoboard** — Placa de pruebas sin soldadura para prototipar circuitos.

**Jumper** — Cable con conector para unir puntos de la protoboard o pines.

**Servo** — Motor con control de posición (0–180°) mediante pulsos; aquí se maneja con Servo.write().

**USB / VIN / 5V** — Formas de alimentar el Arduino: por el puerto USB, por VIN (7–12 V) o directamente 5 V en el pin 5V.

**GND** — Tierra / 0 V, referencia común de todo el circuito. Todos los pines GND de una placa son el mismo nodo.

# 9. Estructura del repositorio y cómo se genera este manual

```
kits_robotica/
 index.html            # página única (explorador de kits + laboratorio)
 css/  styles.css, lab.css
 js/   data.js (los 3 kits) - app.js (navegación)
    lab/  catalog.js (25 componentes y pines) - engine.js (motor MNA)
            mcu.js (intérprete Arduino/ESP32) - projects.js (11 prácticas) - ui.js
 assets/images/        # 33 SVG de componentes + 3 SVG de cajas de kits
 tests/engine.test.js  # 46 pruebas del motor (node tests/engine.test.js)
 tools/export_data.js  # vuelca los datos reales a tools/manual_data.json
 tools/build_manual.py # genera docs/MANUAL.pdf (este documento)
 docs/MANUAL.pdf       # manual de usuario en PDF
```

Para regenerar este PDF después de cambiar datos del proyecto:

```
node tools/export_data.js
python3 tools/build_manual.py   # requiere: pip install reportlab svglib
```

Documento generado el 2026-11-02 — https://github.com/migulopez/kits_robotica — https://migulopez.github.io/kits_robotica/